

UNIVERSIDAD DEL BOSQUE  
Facultad de Ingeniería  
Ingeniería de Software & Ingeniería en Robótica

---

# Competitive Programming Notebook

*Algoritmos, estructuras de datos y plantillas*

C++

---

**Juan Carlos Morales**  
Doble titulación: Sistemas & Robótica

Bogotá, Colombia · 2026

# Índice

|            |                                                          |           |
|------------|----------------------------------------------------------|-----------|
| <b>I</b>   | <b>Introduccion</b>                                      | <b>3</b>  |
| <b>1</b>   | <b>Fundamentos Básicos</b>                               | <b>3</b>  |
| 1.1        | Entrada / Salida Rápida (Fast I/O)                       | 3         |
| 1.2        | Formateo de Salida                                       | 3         |
| 1.3        | Condicionales y Ciclos                                   | 4         |
| 1.4        | Conversiones, Parseos y Casteos                          | 4         |
| <b>II</b>  | <b>Estructuras de Datos</b>                              | <b>5</b>  |
| <b>2</b>   | <b>Basic Structures</b>                                  | <b>5</b>  |
| 2.1        | Matrices (Arreglos 2D)                                   | 5         |
| 2.2        | Vectores / Listas dinámicas                              | 5         |
| 2.3        | Listas enlazadas (LinkedList)                            | 5         |
| 2.4        | Pilas (Stack)                                            | 5         |
| 2.5        | Colas (Queue)                                            | 5         |
| 2.6        | Colas de prioridad (Priority Queue / Heap)               | 6         |
| 2.7        | Vector circular (Circular Buffer)                        | 6         |
| <b>3</b>   | <b>Estructuras Hash/Tree</b>                             | <b>7</b>  |
| 3.1        | Conjuntos hash (HashSet)                                 | 7         |
| 3.2        | Mapas / Diccionarios hash                                | 7         |
| 3.3        | Conjuntos ordenados (TreeSet)                            | 8         |
| 3.4        | Mapas ordenados (TreeMap)                                | 8         |
| <b>4</b>   | <b>Trees</b>                                             | <b>9</b>  |
| <b>5</b>   | <b>Range Queries</b>                                     | <b>9</b>  |
| 5.1        | Casos de uso                                             | 9         |
| 5.2        | Prefix Sums                                              | 10        |
| 5.3        | Árbol de Fenwick (BIT)                                   | 12        |
| 5.4        | Sqrt Decomposition                                       | 13        |
| 5.5        | Sparse Table (RMQ)                                       | 15        |
| 5.6        | Segment Tree                                             | 17        |
| 5.7        | RMQ (Range Minimum Query)                                | 19        |
| 5.8        | Mo's Algorithm                                           | 20        |
| 5.9        | Wavelet Tree                                             | 22        |
| 5.10       | Heavy-Light Decomposition                                | 23        |
| <b>III</b> | <b>Grafos y Árboles</b>                                  | <b>24</b> |
| <b>6</b>   | <b>Grafos (Graphs)</b>                                   | <b>24</b> |
| <b>7</b>   | <b>Árboles (Trees)</b>                                   | <b>25</b> |
| <b>8</b>   | <b>Disjoint Set Union (DSU)</b>                          | <b>25</b> |
| <b>IV</b>  | <b>Ordenamiento y Búsqueda</b>                           | <b>25</b> |
| <b>9</b>   | <b>Ordenamiento y Búsqueda (Sorting &amp; Searching)</b> | <b>25</b> |
| 9.1        | Número faltante (Missing Number)                         | 25        |
| 9.2        | Máximo / Mínimo (Array Max/Min)                          | 25        |
| 9.3        | Par de suma absoluta mínima                              | 25        |
| 9.4        | Par con diferencia dada (Difference Pair)                | 25        |
| 9.5        | Búsqueda ternaria (Ternary Search)                       | 25        |
| 9.6        | Búsqueda de Fibonacci (Fibonacci Search)                 | 25        |
| 9.7        | Búsqueda exponencial (Exponential Search)                | 25        |

|            |                                             |           |
|------------|---------------------------------------------|-----------|
| 9.8        | Búsqueda por saltos (Jump Search)           | 26        |
| 9.9        | Heap Sort                                   | 26        |
| 9.10       | QuickSelect                                 | 26        |
| <b>10</b>  | <b>Búsqueda Binaria (Binary Search)</b>     | <b>26</b> |
| 10.1       | En arreglo ordenado                         | 26        |
| 10.2       | Menor $x$ que cumple                        | 26        |
| 10.3       | Mayor $x$ que cumple                        | 26        |
| 10.4       | Sobre reales                                | 27        |
| 10.5       | Con strings                                 | 27        |
| <b>V</b>   | <b>Matemáticas</b>                          | <b>27</b> |
| 11         | Matemáticas (Math)                          | 27        |
| 12         | Teoría de Números (Number Theory)           | 27        |
| 13         | Combinatoria (Combinatorics)                | 27        |
| 14         | Máscaras de Bits (Bitmasking)               | 27        |
| 15         | Geometría (Geometry)                        | 27        |
| <b>VI</b>  | <b>Técnicas Algorítmicas</b>                | <b>27</b> |
| 16         | Programación Dinámica (Dynamic Programming) | 27        |
| 17         | Algoritmos Voraces (Greedy)                 | 27        |
| 18         | Ad-hoc y Simulación                         | 27        |
| <b>VII</b> | <b>Cadenas</b>                              | <b>27</b> |
| 19         | Cadenas (Strings)                           | 27        |

# Parte I Introduccion

## 1 Fundamentos Básicos

### 1.1 Entrada / Salida Rápida (Fast I/O)

Usar lectura rápida para evitar TLE, hay 3 niveles de lectura rápida, estandarizar Tier 2 y usar Tier 3 (lector por bytes) solo cuando haya del orden de  $10^6$ – $10^7$  lecturas.

Table 1: Casos donde se recomienda Fast I/O

| Tipo de problema                | Ejemplo                                                          |
|---------------------------------|------------------------------------------------------------------|
| Arrays gigantes                 | Leer/ordenar/sumar $10^6$ – $10^7$ enteros                       |
| Muchos test cases (T grande)    | $T \leq 10^5$ , cada uno trivial pero se acumula                 |
| Grafos grandes                  | $M \sim 10^6$ – $2 \cdot 10^6$ aristas para BFS/DFS/Dijkstra/MST |
| Geometria con muchas consultas  | $L, S$ grandes + muchos test cases seguidos hasta EOF            |
| Muchos tokens de texto          | Strings/lineas que suman millones de caracteres                  |
| Judges clasicos "trampa de I/O" | SPOJ INTEST, ARRAY-SUB, etc.                                     |

Regla practica: si  $N/M$  es del orden de  $10^4$  –  $10^5$ , cin con `sync_with_stdio(false); cin.tie(nullptr);` alcanza de sobra. El FastReader es la "artilleria pesada" para cuando eso deja de ser suficiente.

**Tier 1 — Básico:** cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

**Tier 2 — Rápido:** desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

**Tier 3 — Más rápido:** lector por bytes con fread. Solo con  $10^6$ – $10^7$  lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leidos = 0;
5
6     int leer() {
7         if (puntero == leidos) {
8             leidos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leidos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13
14    int nextInt() {
15        int resultado = 0, b = leer();
16        while (b <= ' ') b = leer(); // saltar blancos
17        bool negativo = (b == '-');
18        if (negativo) b = leer();
19        while (b >= '0' && b <= '9') {
20            resultado = resultado * 10 + (b - '0'); b = leer();
21        }
22        return negativo ? -resultado : resultado;
23    }
24 }
```

```
23 long long nextLong() {
24     long long resultado = 0; int b = leer();
25     while (b <= ' ') b = leer();
26     bool negativo = (b == '-');
27     if (negativo) b = leer();
28     while (b >= '0' && b <= '9') {
29         resultado = resultado * 10 + (b - '0'); b = leer();
30     }
31     return negativo ? -resultado : resultado;
32 }
33
34 double nextDouble() {
35     double resultado = 0, divisor = 1; int b = leer();
36     while (b <= ' ') b = leer();
37     bool negativo = (b == '-');
38     if (negativo) b = leer();
39     while (b >= '0' && b <= '9') {
40         resultado = resultado * 10 + (b - '0'); b = leer();
41     }
42     if (b == '.') {
43         b = leer();
44         while (b >= '0' && b <= '9') {
45             resultado += (b - '0') / (divisor *= 10); b = leer();
46         }
47     }
48     return negativo ? -resultado : resultado;
49 }
50
51 string next() { // un token
52     string s; int b = leer();
53     while (b <= ' ') b = leer();
54     while (b > ' ') { s += (char) b; b = leer(); }
55     return s;
56 }
57
58 string nextLine() { // línea completa
59     string s; int b = leer();
60     while (b != '\n' && b != -1) {
61         if (b != '\r') s += (char) b; b = leer();
62     }
63     return s;
64 }
65 }
```

### 1.2 Formateo de Salida

La función `printf` utiliza una cadena de formato para indicar cómo debe interpretarse y mostrarse cada argumento. Cada especificador comienza con `%` y corresponde al argumento que aparece después de la cadena.

```
1 // %d -> entero (int)
2 printf("%d\n", 42);
3 // Entrada: 42
4 // Salida: 42
5
6 // %lld -> entero largo (long long)
7 long long x = 1234567890123LL;
8 printf("%lld\n", x);
9 // Entrada: 1234567890123
10 // Salida: 1234567890123
11
12 // %5d -> ancho mínimo de 5 caracteres, alineado a la derecha
13 printf("%5d\n", 42);
14 // Entrada: 42
15 // Salida: "  42"
16
17 // %-5d -> ancho mínimo de 5 caracteres, alineado a la izquierda
18 printf("%-5d\n", 42);
19 // Entrada: 42
20 // Salida: "42  "
21
22 // %05d -> ancho 5, completando con ceros
23 printf("%05d\n", 42);
24 // Entrada: 42
25 // Salida: "00042"
26
27 // %+d -> muestra siempre el signo
28 printf("%+d\n", 42);
29 // Entrada: 42
30 // Salida: "+42"
31
32 // %x -> hexadecimal usando letras minúsculas
33 printf("%x\n", 255);
34 // Entrada: 255
35 // Salida: ff
36
37 // %X -> hexadecimal usando letras mayúsculas
38 printf("%X\n", 255);
39 // Entrada: 255
40 // Salida: FF
41 }
```

```

42 // %.2f -> numero real con exactamente 2 decimales
43 printf("%.2f\n", 3.14159);
44 // Entrada: 3.14159
45 // Salida: 3.14
46
47 // %.0f -> numero real sin decimales (redondea)
48 printf("%.0f\n", 3.7);
49 // Entrada: 3.7
50 // Salida: 4
51
52 // %e -> notacion cientifica
53 printf("%e\n", 31415.9);
54 // Entrada: 31415.9
55 // Salida: 3.141590e+04
56
57 // %c -> un solo caracter
58 printf("%c\n", 'A');
59 // Entrada: 'A'
60 // Salida: A
61
62 // %s -> cadena de caracteres
63 printf("%s\n", "hola");
64 // Entrada: "hola"
65 // Salida: hola
66
67 // Se pueden combinar varios especificadores
68 printf("%d %lld %.2f\n", 42, 10000000000LL, 3.14159);
69 // Entrada: 42, 10000000000, 3.14159
70 // Salida: 42 10000000000 3.14
71
72 // %% -> imprime un '%' literal
73 printf("100%%\n");
74 // Entrada: no recibe ningun argumento para %
75 // Salida: 100%
76
77 // ! NO uses %n en C/C++ (puede ser peligroso)
78 // Para scanf, un double se lee con %lf

```

## Estructura de un especificador

Un formato puede combinar varias partes:

```

1 // % [flags] [width] [.precision] [specifier]
2 // % 0 5 .2 f
3
4 printf("%05d\n", 42);
5 // % -> comienza el formato
6 // 0 -> rellena con ceros
7 // 5 -> ancho minimo de 5 caracteres
8 // d -> interpreta el argumento como int
9 //
10 // Salida: 00042

```

Los modificadores más utilizados en programación competitiva son:

| Formato | Tipo         | Ejemplo de salida |
|---------|--------------|-------------------|
| %d      | int          | 42                |
| %lld    | long long    | 1234567890123     |
| %c      | char         | A                 |
| %s      | string/char* | hola              |
| %f      | double       | 3.140000          |
| %.2f    | double       | 3.14              |
| %x      | int          | ff                |
| %X      | int          | FF                |

**Regla práctica:** en programación competitiva, los formatos más importantes son %d para int, %lld para long long, %c para caracteres, %s para cadenas y %.2f (o una precisión diferente) para números reales.

## 1.3 Condicionales y Ciclos

```

1 // if / else: ejecuta un bloque segun una condicion
2 if (x > 0) cout << "pos"; // x > 0 -> "pos"
3 else if (x < 0) cout << "neg"; // x < 0 -> "neg"
4 else cout << "cero"; // x == 0 -> "cero"
5
6 // Operador ternario: condicion ? valor_si_true : valor_si_false
7 string s = (x >= 0) ? "no neg" : "neg";
8 // Si x >= 0, s = "no neg"; si no, s = "neg"

```

```

9
10 // switch: compara una expresion con diferentes valores
11 // Solo funciona directamente con tipos integrales como int o
    char,
12 // NO con string.
13 switch (x) {
14     case 1: /*...*/ break; // x == 1
15     case 2: case 3: /*...*/ break; // x == 2 o x == 3
16     default: /*...*/; // ningun case coincide
17 }
18 // ! Sin break, se continua ejecutando el siguiente case (fall-through)
19
20 // C++17: se puede declarar una variable dentro del if
21 if (int r = x % 2; r == 0) {}
22 // r solo existe dentro del if y sus bloques asociados
23
24 // for clasico: usa un indice y se repite mientras i < n
25 for (int i = 0; i < n; i++) {}
26
27 // range-based for: recorre directamente los elementos
28 // No proporciona el indice.
29 for (int v : datos) {}
30
31 // & obtiene una referencia al elemento original.
32 // Por tanto, las modificaciones afectan al vector.
33 for (int &v : datos) v *= 2;
34
35 // while: ejecuta mientras la condicion sea verdadera
36 while (c < n) c++;
37
38 // do-while: primero ejecuta el bloque y DESPUES verifica la
    condicion.
39 // Por eso se ejecuta AL MENOS una vez.
40 do {
41     c--;
42 } while (c > 0);
43
44 // break -> termina inmediatamente el ciclo o switch
45 // continue -> salta a la siguiente iteracion del ciclo
46
47 // ! C++ NO tiene break con etiqueta.
48 // Para salir de ciclos anidados se puede usar una bandera,
49 // encapsular la logica en una funcion y usar return, etc.

```

## 1.4 Conversiones, Parseos y Casteos

```

1 // string -> numero (parseo)
2 // Convierte una cadena de texto en el tipo numerico indicado.
3 int e = stoi("42");
4 // Entrada: "42" -> Salida: 42
5
6 long long l = stoll("10000000000");
7 // Entrada: "10000000000" -> Salida: 10000000000
8
9 double d = stod("3.14");
10 // Entrada: "3.14" -> Salida: 3.14
11
12 // Tambien se puede indicar la base numerica.
13 int bin = stoi("1010", nullptr, 2);
14 // "1010" en base 2 -> 10
15
16 int hex = stoi("ff", nullptr, 16);
17 // "ff" en base 16 -> 255
18
19
20 // numero -> string
21 // Convierte cualquier numero compatible en texto.
22 string s = to_string(42);
23 // Entrada: 42 -> Salida: "42"
24
25
26 // Casteo: fuerza la conversion de un tipo a otro.
27 // Al pasar de decimal a entero, se TRUNCa hacia 0.
28 int t = (int) 3.99;
29 // 3.99 -> 3
30
31 int r = static_cast<int>(3.99);
32 // 3.99 -> 3
33 // static_cast<int> es la forma recomendada en C++
34
35
36 // ! int / int = division ENTERA
37 double mal = 7 / 2;
38 // 7 y 2 son int -> 3
39 // Luego 3 se convierte a double -> 3.0
40
41 double bien = 7 / 2.0;
42 // 2.0 es double -> division decimal
43 // 7 / 2.0 -> 3.5
44

```

```

45 // caracteres
46 // Los char tienen un valor numerico asociado (ASCII).
47 int cod = (int) 'A';
48 // 'A' -> 65
49
50
51 char ch = (char) 66;
52 // 66 -> 'B'
53
54 // Truco muy usado para convertir un digito char a int.
55 int dig = '7' - '0';
56 // '7' tiene valor 55 y '0' tiene valor 48
57 // 55 - 48 -> 7

```

## Parte II Estructuras de Datos

### 2 Basic Structures

```

1 int a[5] = {1,2,3,4,5}; // crear con valores: O(n)
2
3 int b[5];                // crear de tamaño 5
4                          // posiciones: b[0] ... b[4]
5                          // ! Valores no inicializados
6
7 b[0] = 10;               // INSERT: O(1)
8 b[1] = 20;               // INSERT: O(1)
9
10 int x = b[0];            // READ: O(1)
11
12 b[0] = 30;               // UPDATE: O(1)
13
14 for (int i = 0; i < 5; i++)
15     cout << b[i] << " "; // READ: recorrer -> O(n)
16
17 // DELETE: no existe metodo para eliminar
18 // Se puede sobrescribir o manejar logicamente
19 b[0] = 0;                // DELETE logico: O(1)
20
21 // ! El tamaño de un array estatico no cambia.
22 // Para mas posiciones se necesita otro arreglo.

```

#### 2.1 Matrices (Arreglos 2D)

Arreglo bidimensional (filas  $\times$  columnas). **Complejidad:** acceso  $[i][j]$   $O(1)$ ; crear  $n \times m$  en  $O(nm)$ .

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // CREATE: O(n*m)
2 // matriz 3x4, todas las posiciones inicializadas en 0
3
4 m[1][2] = 7;             // INSERT: O(1)
5
6 int x = m[1][2];         // READ: O(1)
7
8 m[1][2] = 10;            // UPDATE: O(1)
9
10 for (int i = 0; i < 3; i++)
11     for (int j = 0; j < 4; j++)
12         cout << m[i][j] << " ";
13 // READ: recorrer toda la matriz -> O(n*m)
14
15 // DELETE: no existe metodo para eliminar una posicion.
16 // Se puede sobrescribir el valor o manejarlo logicamente.
17 m[1][2] = 0;             // DELETE logico: O(1)
18
19 // ! El tamaño de la matriz puede cambiar con vector,
20 // pero un arreglo 2D estatico no puede cambiar de tamaño.

```

#### 2.2 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice  $O(1)$ ; agregar al final  $O(1)$  amortizado; insertar/borrar en medio  $O(n)$ ; buscar  $O(n)$ .

```

1 vector<int> v;            // CREATE: lista vacia
2
3 v.push_back(3);          // INSERT: final -> O(1) amortizado
4 v.push_back(7);          // INSERT: final -> O(1) amortizado
5
6 int x = v[0];            // READ: acceso -> O(1)

```

```

7 int n = v.size();        // READ: tamaño -> O(1)
8
9 v[0] = 10;               // UPDATE: modificar -> O(1)
10
11 v.insert(v.begin() + 1, 5); // INSERT: medio -> O(n)
12
13 v.erase(v.begin() + 1);  // DELETE: medio -> O(n)
14
15 // Buscar un elemento recorriendo el vector
16 for (int x : v)
17     if (x == 10) { /* ... */ }
18 // SEARCH: recorrer -> O(n)
19
20 // DELETE: eliminar el ultimo elemento
21 v.pop_back();            // DELETE: final -> O(1)
22
23 // ! vector mantiene acceso por índice O(1).
24 // ! Al crecer puede realocar su memoria internamente.

```

#### 2.3 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos**  $O(1)$ ; acceso por índice  $O(n)$ ; buscar  $O(n)$ . Útil como cola o deque.

```

1 list<int> l;              // CREATE: lista vacia
2
3 l.push_front(1);          // INSERT: inicio -> O(1)
4 l.push_back(9);           // INSERT: final -> O(1)
5
6 int x = l.front();        // READ: primer elemento -> O(1)
7 int y = l.back();         // READ: ultimo elemento -> O(1)
8
9 l.front() = 5;            // UPDATE: primer elemento -> O(1)
10 l.back() = 10;           // UPDATE: ultimo elemento -> O(1)
11
12 l.pop_front();            // DELETE: inicio -> O(1)
13 l.pop_back();             // DELETE: final -> O(1)
14
15 // ! No existe acceso directo por índice.
16 // Avanzar hasta una posición requiere recorrer nodos.
17 auto it = l.begin();
18 advance(it, 2);           // acceso al tercer elemento -> O(n)
19
20 // Buscar un elemento requiere recorrer la lista.
21 for (int x : l)
22     if (x == 10) { /* ... */ }
23 // SEARCH: recorrer -> O(n)
24
25 // ! list no permite l[i].
26 // Para acceso por índice, vector suele ser mejor.

```

#### 2.4 Pilas (Stack)

Estructura **LIFO** (último en entrar, primero en salir). **Complejidad:** insertar, eliminar y consultar el tope  $O(1)$ . Para DFS iterativo, paréntesis balanceados y deshacer.

```

1 stack<int> pila;         // CREATE: pila vacia
2
3 pila.push(3);            // INSERT: O(1)
4 pila.push(7);            // INSERT: O(1)
5
6 int tope = pila.top();    // READ: consultar tope -> O(1)
7
8 pila.top() = 10;         // UPDATE: modificar tope -> O(1)
9
10 pila.pop();              // DELETE: eliminar tope -> O(1)
11
12 // ! Solo se puede acceder directamente al elemento superior.
13 // ! No existe acceso por índice.

```

#### 2.5 Colas (Queue)

Estructura **FIFO** (primero en entrar, primero en salir). **Complejidad:** encolar y desencolar  $O(1)$ . Base del BFS.

```

1 queue<int> cola;         // CREATE: cola vacia
2
3 cola.push(3);            // INSERT: final -> O(1)

```

```

4 cola.push(7);           // INSERT: final -> O(1)
5
6 int frente = cola.front(); // READ: primero -> O(1)
7 int atras = cola.back();  // READ: ultimo -> O(1)
8
9 cola.front() = 10;       // UPDATE: modificar primero -> O(1)
10
11 cola.pop();             // DELETE: primero -> O(1)
12
13 // ! push agrega al final y pop elimina del frente.
14 // ! No existe acceso directo por índice.

```

## 2.6 Colas de prioridad (Priority Queue / Heap)

Devuelve siempre el elemento de mayor (o menor) prioridad. **Complejidad:** insertar y extraer  $O(\log n)$ ; consultar el tope  $O(1)$ . Para Dijkstra, Prim, k-ésimo mayor.

```

1 priority_queue<int> pq; // CREATE: MAX-heap por defecto
2
3 pq.push(3);           // INSERT: O(log n)
4 pq.push(7);           // INSERT: O(log n)
5
6 int mx = pq.top();     // READ: mayor -> O(1)
7
8 pq.pop();             // DELETE: mayor -> O(log n)
9
10 // min-heap: el menor queda en el tope
11 priority_queue<int, vector<int>, greater<int>> pqMin;
12
13 pqMin.push(3);        // INSERT: O(log n)
14 int mn = pqMin.top(); // READ: menor -> O(1)
15 pqMin.pop();          // DELETE: O(log n)
16
17 // ! No existe acceso directo a elementos intermedios.
18 // ! priority_queue no tiene una operacion UPDATE directa.

```

## 2.7 Vector circular (Circular Buffer)

**Idea:** un arreglo de tamaño fijo (*cap*) que se comporta como una **cola doble**. En lugar de mover elementos cuando se inserta o elimina, se reutilizan las posiciones del arreglo de forma circular.

Se mantienen tres variables principales:

- *cap*: capacidad total del arreglo.
- *ini*: posición física donde comienza el primer elemento.
- *tam*: cantidad actual de elementos almacenados.

La posición lógica *i* no coincide necesariamente con la posición física del arreglo. Se obtiene mediante:

$$\text{posicionFisica}(i) = (\text{ini} + i) \% \text{cap}$$

Por ejemplo, con *cap* = 5 y *ini* = 3:

| Índice lógico | 0 | 1 | 2 | 3 | 4 |
|---------------|---|---|---|---|---|
| Índice físico | 3 | 4 | 0 | 1 | 2 |

Cuando se llega al final del arreglo, el módulo hace que la siguiente posición vuelva al comienzo. Por eso **no es necesario mover los elementos**.

¿Por qué usarlo? En un arreglo normal, insertar al frente requiere desplazar todos los elementos y cuesta  $O(n)$ . En un vector circular solo se mueve *ini*, por lo que cuesta  $O(1)$ .

**Casos de uso:**

- Colas de tamaño fijo.
- Buffers de audio, video o red.

- Ventanas deslizantes.
- Sistemas donde llegan y salen datos continuamente.
- Simulaciones de procesos en una cola.
- Mantener únicamente los últimos *k* elementos.

Si el buffer está lleno y se ejecuta **pushBack**, el nuevo elemento **reemplaza al más antiguo**. Esto permite mantener automáticamente una ventana de tamaño fijo.

**Complejidad:** insertar, eliminar, consultar extremos, acceder por posición y comprobar el tamaño son  $O(1)$ .

```

1 struct VectorCircular {
2     vector<int> buf;
3     int cap, ini = 0, tam = 0;
4
5     VectorCircular(int c) : buf(c), cap(c) {}
6
7     // CREATE: crea un buffer con capacidad fija.
8     // Los elementos inicialmente no estan ocupados.
9
10    bool empty() {
11        return tam == 0;
12    } // O(1)
13
14    bool full() {
15        return tam == cap;
16    } // O(1)
17
18    int size() {
19        return tam;
20    } // O(1)
21
22    int capacity() {
23        return cap;
24    } // O(1)
25
26    // Convierte una posicion logica en fisica.
27    int pos(int i) {
28        return (ini + i) % cap;
29    } // O(1)
30
31    // INSERT: agrega al final.
32    void pushBack(int x) {
33        buf[(ini + tam) % cap] = x;
34
35        if (tam < cap)
36            tam++;
37        else
38            ini = (ini + 1) % cap;
39    } // O(1)
40
41    // INSERT: agrega al inicio.
42    void pushFront(int x) {
43        ini = (ini - 1 + cap) % cap;
44        buf[ini] = x;
45
46        if (tam < cap)
47            tam++;
48    } // O(1)
49
50    // READ: consulta el primer elemento.
51    int front() {
52        return buf[ini];
53    } // O(1)
54
55    // READ: consulta el ultimo elemento.
56    int back() {
57        return buf[(ini + tam - 1) % cap];
58    } // O(1)
59
60    // READ: acceso por posicion logica.
61    int at(int i) {
62        return buf[(ini + i) % cap];
63    } // O(1)
64
65    // UPDATE: modifica un elemento.
66    void set(int i, int x) {
67        buf[(ini + i) % cap] = x;
68    } // O(1)
69
70    // DELETE: elimina el primer elemento.
71    int popFront() {
72        int v = buf[ini];
73        ini = (ini + 1) % cap;
74        tam--;
75        return v;
76    } // O(1)

```



```

77 // DELETE: elimina el ultimo elemento.
78 int popBack() {
79     tam--;
80     return buf[(ini + tam) % cap];
81 } // O(1)
82
83 // DELETE: elimina todos los elementos.
84 void clear() {
85     ini = 0;
86     tam = 0;
87 } // O(1)
88
89 // SEARCH: busca un elemento.
90 // A diferencia de las operaciones anteriores,
91 // hay que recorrer los elementos.
92 bool contains(int x) {
93     for (int i = 0; i < tam; i++)
94         if (at(i) == x)
95             return true;
96     return false;
97 } // O(n)
98
99 // READ: recorrer los elementos en orden logico.
100 void print() {
101     for (int i = 0; i < tam; i++)
102         cout << at(i) << " ";
103 } // O(n)
104 };
105
106

```

### Ejemplo de funcionamiento:

```

1 VectorCircular v(5);
2
3 v.pushBack(10);
4 v.pushBack(20);
5 v.pushBack(30);
6 // [10, 20, 30, _, _]
7 // ini = 0, tam = 3
8
9 v.pushFront(5);
10 // [5, 10, 20, 30, _]
11 // ini cambia de 0 a 4
12
13 cout << v.front();
14 // 5
15
16 cout << v.back();
17 // 30
18
19 v.popFront();
20 // elimina 5
21 // [_, 10, 20, 30, _]
22
23 v.pushBack(40);
24 v.pushBack(50);
25 // [_, 10, 20, 30, 40, 50]
26 // El almacenamiento puede haber dado la vuelta.
27
28 v.pushBack(60);
29 // El buffer estaba lleno.
30 // Se agrega 60 y se elimina el mas antiguo.
31 // Resultado logico: [20, 30, 40, 50, 60]

```

**Ventaja principal:** el arreglo físico puede verse “desordenado”, pero los elementos siempre tienen un orden lógico determinado por `ini`. Por ejemplo:

```

1 // Memoria fisica:
2 // [40, 50, 20, 30, _]
3 //      ^
4 //      ini = 2
5
6 // Orden logico:
7 // 20 -> 30 -> 40 -> 50

```

No importa dónde estén físicamente los elementos: `at(0)` siempre representa el primero, `at(1)` el segundo, y así sucesivamente.

### Diferencia con un vector normal:

| Operación          | vector   | Circular |
|--------------------|----------|----------|
| Agregar al final   | $O(1)^*$ | $O(1)$   |
| Eliminar al final  | $O(1)$   | $O(1)$   |
| Agregar al frente  | $O(n)$   | $O(1)$   |
| Eliminar al frente | $O(n)$   | $O(1)$   |
| Acceso por índice  | $O(1)$   | $O(1)$   |
| Buscar             | $O(n)$   | $O(n)$   |

\*Amortizado.

**Regla práctica:** si necesitas una estructura de tamaño fijo donde los elementos entran y salen continuamente por los extremos, un vector circular evita los desplazamientos de un arreglo normal y mantiene las operaciones principales en  $O(1)$ .

## 3 Estructuras Hash/Tree

### 3.1 Conjuntos hash (HashSet)

Colección de elementos **sin duplicados** y sin orden garantizado. En C++ se implementa mediante `unordered_set`. Utiliza una tabla hash, por lo que las operaciones son  $O(1)$  en promedio. Es útil cuando solo interesa saber si un elemento existe, sin necesidad de mantener los elementos ordenados.

**Complejidad:** insertar, buscar y borrar  $O(1)$  promedio ( $O(n)$  peor caso).

```

1 unordered_set<int> s; // CREATE: conjunto vacio
2
3 s.insert(3); // INSERT: O(1) prom.
4 s.insert(7); // INSERT: O(1) prom.
5 s.insert(3); // no se agrega: ya existe
6
7 bool hay = s.count(3); // SEARCH: O(1) prom.
8 // hay = 1 si existe, 0 si no existe
9
10 auto it = s.find(7); // SEARCH: O(1) prom.
11 // devuelve iterador al elemento o s.end()
12
13 s.erase(3); // DELETE: O(1) prom.
14
15 int n = s.size(); // READ: cantidad -> O(1)
16 bool vacio = s.empty(); // READ: ¿esta vacio? -> O(1)
17
18 // Recorrer todos los elementos
19 for (int x : s)
20     cout << x << " ";
21 // O(n), pero el orden NO esta garantizado.
22
23 // ! No existe acceso por indice: s[0] no es valido.
24 // ! Los elementos repetidos se ignoran.
25 // ! El orden puede cambiar entre ejecuciones.

```

### Cuándo usarlo:

- Saber rápidamente si un elemento ya apareció.
- Eliminar duplicados.
- Mantener un conjunto de elementos visitados.
- Detectar elementos repetidos en  $O(n)$  promedio.
- BFS/DFS: guardar vértices visitados cuando sea conveniente.

### 3.2 Mapas / Diccionarios hash

Estructura que almacena pares **clave** → **valor**. En C++ se implementa mediante `unordered_map`. Cada clave identifica un valor y **no puede repetirse**. Si se asigna un nuevo valor a una clave existente, se reemplaza el anterior.

**Complejidad:** insertar, acceder, buscar y borrar por clave  $O(1)$  promedio ( $O(n)$  peor caso).



```

1 unordered_map<string,int> m; // CREATE: mapa vacio
2
3 m["a"] = 1; // INSERT: O(1) prom.
4 m["b"] = 2; // INSERT: O(1) prom.
5
6 int v = m["a"]; // READ: acceder -> O(1) prom.
7 // v = 1
8
9 m["a"] = 10; // UPDATE: O(1) prom.
10 // la clave "a" ahora tiene valor 10
11
12 m.erase("a"); // DELETE: O(1) prom.
13
14 bool existe = m.count("b"); // SEARCH: O(1) prom.
15 // existe = 1
16
17 auto it = m.find("b"); // SEARCH: O(1) prom.
18 if (it != m.end())
19     cout << it->second; // valor asociado a "b"
20
21 // Recorrer todas las parejas
22 for (auto [clave, valor] : m)
23     cout << clave << " " << valor;
24 // O(n), sin orden garantizado.

```

**Importante:** usar `m[clave]` para consultar una clave que no existe puede **crear esa clave** con su valor por defecto. Para comprobar existencia sin insertar, es preferible usar `count()` o `find()`.

```

1 unordered_map<string,int> m;
2
3 int x = m["a"];
4 // Si "a" no existia, se crea:
5 // "a" -> 0
6 // ! operator[] puede INSERTAR una clave al consultar.
7
8 // count(): comprueba si existe SIN insertar
9 bool existe = m.count("b");
10 // Devuelve 0 si no existe, 1 si existe.
11
12 // find(): busca la clave SIN insertar
13 auto it = m.find("b");
14
15 if (it != m.end())
16     cout << it->second; // valor asociado a "b"
17 else
18     cout << "No existe";
19
20 // ! count() solo indica si existe.
21 // ! find() permite acceder a la pareja clave -> valor.

```

#### Uso clásico: contar frecuencias.

```

1 unordered_map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++; // frecuencia de cada elemento
5
6 cout << freq[5];
7 // cantidad de veces que aparece 5

```

#### Cuándo usarlo:

- Contar frecuencias.
- Asociar identificadores con información.
- Buscar rápidamente un valor a partir de una clave.
- Problemas de conteo y frecuencia.
- Memorizar resultados de estados o funciones.

### 3.3 Conjuntos ordenados (TreeSet)

Conjunto de elementos **sin duplicados y ordenados**. En C++ se implementa mediante `set`, normalmente como un árbol balanceado.

A diferencia de `unordered_set`, los elementos siempre se mantienen ordenados según el comparador utilizado.

**Complejidad:** insertar, buscar y borrar  $O(\log n)$ . Consultar el menor o mayor elemento mediante `begin()` o `rbegin()` es  $O(1)$ .

```

1 set<int> s; // CREATE: conjunto vacio
2
3 s.insert(5); // INSERT: O(log n)
4 s.insert(1); // INSERT: O(log n)
5 s.insert(3); // INSERT: O(log n)
6 s.insert(3); // no se agrega: ya existe
7
8 int menor = *s.begin(); // READ: minimo -> O(1)
9 int mayor = *s.rbegin(); // READ: maximo -> O(1)
10
11 auto it = s.find(3); // SEARCH: O(log n)
12 if (it != s.end())
13     cout << *it;
14
15 s.erase(3); // DELETE: O(log n)
16
17 int n = s.size(); // READ: cantidad -> O(1)
18
19 // Recorrer en orden ascendente
20 for (int x : s)
21     cout << x << " ";
22 // O(n)

```

#### lower\_bound y upper\_bound:

```

1 set<int> s = {1, 3, 5, 7, 9};
2
3 auto it1 = s.lower_bound(5);
4 // primer elemento >= 5
5 // resultado: 5
6
7 auto it2 = s.upper_bound(5);
8 // primer elemento > 5
9 // resultado: 7

```

Esto permite resolver consultas de **piso y techo** rápidamente:

```

1 // FLOOR: mayor elemento <= x
2 auto it = s.upper_bound(x);
3
4 if (it != s.begin()) {
5     --it;
6     cout << *it;
7 }
8
9 // CEILING: menor elemento >= x
10 auto it2 = s.lower_bound(x);
11
12 if (it2 != s.end())
13     cout << *it2;

```

#### Cuándo usarlo:

- Cuando se necesitan elementos únicos y ordenados.
- Obtener mínimo o máximo dinámicamente.
- Encontrar el siguiente elemento mayor o igual.
- Encontrar el anterior elemento menor o igual.
- Mantener un conjunto mientras se insertan y eliminan elementos.

**Diferencia clave:** si solo necesitas saber si existe un elemento, `unordered_set` suele ser más rápido. Si necesitas consultar orden, `lower_bound`, `upper_bound`, `begin()` o `rbegin()`, `set` es la opción adecuada.

### 3.4 Mapas ordenados (TreeMap)

Mapa **clave** → **valor** cuyas claves se mantienen ordenadas. En C++ se implementa mediante `map`, usando un árbol balanceado.

Cada clave aparece una sola vez. Los valores pueden repetirse.

**Complejidad:** insertar, buscar, acceder y borrar por clave  $O(\log n)$ . `begin()` permite obtener la menor clave en  $O(1)$ .

```

1 map<string,int> m;           // CREATE: mapa vacio
2
3 m["b"] = 2;                 // INSERT: O(log n)
4 m["a"] = 1;                 // INSERT: O(log n)
5 m["c"] = 3;                 // INSERT: O(log n)
6
7 int v = m["a"];             // READ: O(log n)
8 // v = 1
9
10 m["a"] = 10;                // UPDATE: O(log n)
11
12 m.erase("b");               // DELETE: O(log n)
13
14 auto it = m.find("c");       // SEARCH: O(log n)
15 if (it != m.end())
16     cout << it->second;
17
18 // Primera pareja: menor clave
19 auto primera = m.begin();    // O(1)
20 cout << primera->first;
21
22 // Ultima pareja: mayor clave
23 auto ultima = prev(m.end()); // O(1) amortizado
24 cout << ultima->first;

```

Las claves se recorren ordenadas:

```

1 map<int,string> m;
2
3 m[30] = "C";
4 m[10] = "A";
5 m[20] = "B";
6
7 for (auto [clave, valor] : m)
8     cout << clave << " " << valor;
9
10 // Salida:
11 // 10 A
12 // 20 B
13 // 30 C

```

`lower_bound` y `upper_bound` también funcionan sobre las claves:

```

1 map<int,string> m;
2 m[10] = "A";
3 m[20] = "B";
4 m[30] = "C";
5
6 auto it = m.lower_bound(15);
7 // primera clave >= 15
8 // apunta a 20
9
10 auto it2 = m.upper_bound(20);
11 // primera clave > 20
12 // apunta a 30

```

Uso clásico: contar frecuencias ordenadas.

```

1 map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;
5
6 // Las claves aparecen en orden creciente.
7 for (auto [x, cantidad] : freq)
8     cout << x << ": " << cantidad << "\n";

```

Cuándo usarlo:

- Cuando se necesitan pares clave-valor ordenados.
- Contar frecuencias y procesarlas en orden.
- Encontrar claves inmediatamente mayores o menores.
- Procesar eventos o valores manteniendo un orden dinámico.
- Cuando se necesitan operaciones de árbol como `lower_bound` y `upper_bound`.

## Comparación rápida

| Estructura                 | Duplicados | Orden | Buscar       |
|----------------------------|------------|-------|--------------|
| <code>unordered_set</code> | No         | No    | $O(1)$ prom. |
| <code>unordered_map</code> | Claves: No | No    | $O(1)$ prom. |
| <code>set</code>           | No         | Sí    | $O(\log n)$  |
| <code>map</code>           | Claves: No | Sí    | $O(\log n)$  |

**Regla práctica:**

- `unordered_set` → ¿Ya existe este elemento?
- `unordered_map` → ¿Qué valor corresponde a esta clave?
- `set` → Elementos únicos + ordenados.
- `map` → Claves → valores + claves ordenadas.

## 4 Trees

## 5 Range Queries

### 5.1 Casos de uso

| Cuando el problema te pide:                                                                                                            | Entonces usa:                                 | Complejidad                              |
|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| Suma de rango sobre un arreglo que <b>nunca cambia</b> , con muchas consultas (ej. "suma de ventas entre el día $l$ y $r$ ")           | Prefix Sums                                   | Build $O(n)$<br>Query $O(1)$             |
| Suma de rango con actualizaciones <b>puntuales</b> frecuentes (ej. "suma el stock de la posición $i$ , luego consulta un rango")       | Fenwick Tree (BIT)                            | Build $O(n)$<br>Query/Update $O(\log n)$ |
| Sumar $v$ a <b>todo un rango</b> $[l, r]$ y también consultar la <b>suma</b> de cualquier rango                                        | FenwickRangeUpdate (doble BIT)                | Query/Update $O(\log n)$                 |
| Range update + range query con operaciones más complejas: min/max en rango, asignar un valor a todo un rango, contar ceros, etc.       | Segment Tree + Lazy Propagation               | Query/Update $O(\log n)$                 |
| Mínimo, máximo o gcd de cualquier rango sobre un arreglo <b>estático</b> (ej. RMQ clásico, LCA)<br>Query $O(1)$                        | Sparse Table                                  | Build $O(n \log n)$                      |
| Mínimo o máximo de rango cuando el arreglo <b>sí cambia</b> con el tiempo                                                              | Segment Tree                                  | Query/Update $O(\log n)$                 |
| Consultas 2D: cuántos puntos o cuánta suma hay dentro de un rectángulo $[x_1, x_2] \times [y_1, y_2]$                                  | Fenwick 2D                                    | Query/Update $O(\log n \log m)$          |
| Encontrar el $k$ -ésimo elemento, o "menor índice cuya suma acumulada alcanza $X$ " (frecuencias)                                      | Fenwick con <code>lowerBound</code>           | $O(\log n)$                              |
| Muchas queries de rango <b>offline</b> (las conoces todas de antemano) sobre un arreglo estático o con pocos updates                   | Mo's Algorithm                                | $O((n+q)\sqrt{n})$                       |
| LCA entre dos nodos, o mínimo en rango sobre un recorrido Euler de un árbol (valores consecutivos difieren en $\pm 1$ )                | RMQ $\pm 1$ (Euler Tour + Sparse Table)       | Build $O(n)$<br>Query $O(1)$             |
| Cuántos elementos en el rango $[l, r]$ están dentro de un rango de <b>valores</b> $[a, b]$ , o cuál es el $k$ -ésimo menor en $[l, r]$ | Wavelet Tree                                  | Query $O(\log n)$                        |
| Queries o updates sobre <b>caminos o subárboles</b> de un árbol (ej. "suma el camino de $u$ a $v$ ")                                   | Heavy-Light Decomposition (+ Segment/Fenwick) | Query/Update $O(\log^2 n)$               |
| Necesitas algo simple de implementar bajo presión, con balance razonable entre update y query, sin memorizar mucha teoría              | Sqrt Decomposition                            | Query/Update $O(\sqrt{n})$               |

Table 2: Guía rápida: qué estructura de range queries usar según el problema, con su complejidad.

## 5.2 Prefix Sums

**Idea:** si un arreglo **no cambia**, puedes precalcular  $\text{pref}[i] = \text{suma de los primeros } i \text{ elementos}$  en  $O(n)$ , una sola vez. Cualquier suma de rango  $[l, r]$  se responde restando dos prefijos:  $\text{pref}[r+1] - \text{pref}[l]$ . Es la base conceptual de todo lo demás en esta sección: Fenwick y Segment Tree existen justamente para soportar lo que Prefix Sums no puede (actualizaciones), a costa de más complejidad de código.

¿Por qué usarlo? Es lo más simple y rápido posible para consultas de rango cuando el arreglo es estático. Cero estructuras de datos elaboradas, cero recursión, solo un arreglo auxiliar y una resta.

### Casos de uso:

- Suma de rango sobre un arreglo que nunca se modifica, con muchas queries.
- Suma de submatriz (versión 2D) sobre una matriz estática.
- Aplicar  $B$  actualizaciones de rango **offline** y consultar el arreglo final una sola vez (Difference Array).
- Como paso final de un precómputo de teoría de números (criba de  $\phi$ ,  $\mu$ , número de divisores, etc.) cuando el problema pide sumas sobre esos valores en un rango.

**Complejidad:** construir  $O(n)$  (u  $O(nm)$  en 2D), consulta  $O(1)$ . No soporta actualizaciones puntuales eficientes: modificar un elemento y volver a consultar obliga a reconstruir,  $O(n)$ .

### Prefix Sum 1D

```
1 struct PrefixSum1D {
2     vector<long long> pref;
3     int n;
4
5     // CREATE: construye los prefijos a partir de a[0..n-1].
6     PrefixSum1D(vector<long long>& a) {
7         n = a.size();
8         pref.assign(n + 1, 0);
9
10        for (int i = 0; i < n; i++)
11            pref[i + 1] = pref[i] + a[i];    // O(n)
12    }
13
14    // READ: suma del rango [l, r] (0-indexado, inclusivo).
15    long long rango(int l, int r) {
16        if (l > r) return 0;
17        return pref[r + 1] - pref[l];    // O(1)
18    }
19 };
```

### Ejemplo de funcionamiento:

```
1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 PrefixSum1D ps(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << ps.rango(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << ps.rango(1, 4);
10 // 2 + 9 + 1 + 7 = 19
11
12 cout << ps.rango(3, 3);
13 // 1
```

### Prefix Sum 2D

```
1 struct PrefixSum2D {
2     vector<vector<long long>> pref;
3     int n, m;
4
5     // CREATE: construye la matriz de prefijos a partir de
6     // a[0..n-1][0..m-1] usando inclusion-exclusion.
7     PrefixSum2D(vector<vector<long long>>& a) {
8         n = a.size();
9         m = a[0].size();
10        pref.assign(n + 1, vector<long long>(m + 1, 0));
11
12        for (int i = 0; i < n; i++)
13            for (int j = 0; j < m; j++)
14                pref[i + 1][j + 1] = a[i][j] + pref[i][j + 1]
15                    + pref[i + 1][j] - pref[i][j];
16    }    // O(n * m)
17
18    // READ: suma del rectangulo [x1..x2] x [y1..y2] (0-indexado,
19    // inclusivo), usando inclusion-exclusion sobre los prefijos.
20    long long rango(int x1, int y1, int x2, int y2) {
21        return pref[x2 + 1][y2 + 1] - pref[x1][y2 + 1]
22            - pref[x2 + 1][y1] + pref[x1][y1];    // O(1)
23    }
24 };
```

### Ejemplo de funcionamiento:

```
1 vector<vector<long long>> a = {
2     {1, 2, 3},
3     {4, 5, 6},
4     {7, 8, 9}
5 };
6
7 PrefixSum2D ps(a);
8
9 cout << ps.rango(0, 0, 1, 1);
10 // 1+2+4+5 = 12
11
12 cout << ps.rango(1, 1, 2, 2);
13 // 5+6+8+9 = 28
```

### Difference Array (updates de rango offline)

**Idea:** el truco inverso a Prefix Sums: para sumar  $v$  a todo un rango  $[l, r]$ , marcas  $\text{diff}[l] += v$  y  $\text{diff}[r+1] -= v$  en  $O(1)$ . Después de aplicar **todas** las actualizaciones, un solo recorrido con prefijos reconstruye el arreglo final. Sirve solo cuando todas las updates son **offline** (las conoces antes de necesitar consultar); si necesitas consultar *entre* actualizaciones, usa Fenwick-RangeUpdate en su lugar.

```
1 struct DifferenceArray {
2     vector<long long> diff;
3     int n;
4
5     // CREATE: reserva un arreglo de diferencias de tamaño n.
6     DifferenceArray(int n) : n(n) {
7         diff.assign(n + 1, 0);
8     }
9
10    // UPDATE: marca +v en todo el rango [l, r] (0-indexado).
11    void rangoUpdate(int l, int r, long long v) {
12        diff[l] += v;
13        if (r + 1 <= n)
14            diff[r + 1] -= v;    // O(1)
15    }
16
17    // READ: reconstruye el arreglo final tras aplicar todas
18    // las actualizaciones marcadas hasta el momento.
19    vector<long long> aplicar() {
20        vector<long long> a(n);
21        long long acc = 0;
22
23        for (int i = 0; i < n; i++) {
24            acc += diff[i];
25            a[i] = acc;
26        }
27
28        return a;    // O(n)
29    }
30 };
```

## Ejemplo de funcionamiento:

```

1 DifferenceArray d(5);
2
3 d.rangoUpdate(1, 3, 4);
4 d.rangoUpdate(0, 4, 1);
5 d.rangoUpdate(2, 2, 10);
6
7 vector<long long> resultado = d.aplicar();
8 // resultado: [1, 5, 15, 5, 1]

```

Aplicación: suma de Euler Totient ( $\phi$ ) en un rango

**Idea:**  $\phi(n)$  cuenta cuántos enteros en  $[1, n]$  son coprimos con  $n$ . Calcular  $\phi$  de un número es teoría de números ( $O(\sqrt{n})$  factorizando); pero cuando el problema pide la suma de  $\phi(i)$  para  $i$  en un rango  $[l, r]$ , con muchas queries, el patrón se resuelve en dos fases: (1) precalculas el arreglo de  $\phi$  una sola vez con una criba (teoría de números), y (2) respondes las queries de rango con Prefix Sums sobre ese arreglo ya calculado (exactamente esta subsección). Factorizar cada número del rango por separado en cada query sería  $O(q\sqrt{n})$  y no escala; precalcular y sumar con prefijos reduce cada query a  $O(1)$ .

Criba lineal + Prefix Sums (rango  $[1, N]$ ).

```

1 struct PhiRangeSum {
2     vector<long long> phi, pref;
3     int n;
4
5     // CREATE: calcula phi[1..n] con criba lineal y sus prefijos.
6     // phi[i] = cantidad de enteros en [1, i] coprimos con i.
7     PhiRangeSum(int n) : n(n) {
8         phi.assign(n + 1, 0);
9         pref.assign(n + 1, 0);
10
11         vector<int> primos;
12         vector<bool> compuesto(n + 1, false);
13         phi[1] = 1;
14
15         for (int i = 2; i <= n; i++) {
16             if (!compuesto[i]) {
17                 primos.push_back(i);
18                 phi[i] = i - 1;
19             }
20
21             for (int p : primos) {
22                 if ((long long)i * p > n) break;
23
24                 compuesto[i * p] = true;
25
26                 if (i % p == 0) {
27                     phi[i * p] = phi[i] * p;
28                     break;
29                 } else {
30                     phi[i * p] = phi[i] * (p - 1);
31                 }
32             }
33         }
34
35         for (int i = 1; i <= n; i++)
36             pref[i] = pref[i - 1] + phi[i]; // O(n)
37     }
38
39     // READ: suma de phi(i) para i en [l, r].
40     long long rango(int l, int r) {
41         if (l > r) return 0;
42         return pref[r] - pref[l - 1]; // O(1)
43     }
44
45     // READ: valor individual de phi(i).
46     long long get(int i) {
47         return phi[i]; // O(1)
48     }
49 };

```

## Ejemplo de funcionamiento:

```

1 PhiRangeSum pr(10);
2 // phi(1..10) = [1, 1, 2, 2, 4, 2, 6, 4, 6, 4]
3
4 cout << pr.get(7);
5 // phi(7) = 6 (7 es primo)

```

```

6 cout << pr.rango(1, 10);
7 // 1+1+2+2+4+2+6+4+6+4 = 32
8
9
10 cout << pr.rango(4, 8);
11 // 2+4+2+6+4 = 18

```

**Criba segmentada (rango  $[L, R]$  grande).** Si  $R$  es muy grande ( $10^{12}$ ) pero  $R - L$  es manejable, no puedes cribar desde 1. En vez de eso, precalculas los primos hasta  $\sqrt{R}$ , y para cada número del segmento  $[L, R]$  le quitas sus factores primos pequeños, aplicando  $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$ . Si al final queda un residuo  $> 1$ , es un único primo grande (mayor a  $\sqrt{R}$ ) que también se aplica a la fórmula.

```

1 struct PhiSegmentedRangeSum {
2     vector<long long> phi, pref;
3     long long L, R;
4
5     // CREATE: calcula phi(i) para cada i en [L, R] factorizando
6     // los primos hasta sqrt(R), y sus prefijos sobre el segmento.
7
8     PhiSegmentedRangeSum(long long L, long long R) : L(L), R(R) {
9         int lim = (int)sqrt((double)R) + 1;
10
11         vector<int> primos;
12         vector<bool> compuesto(lim + 1, false);
13         for (int i = 2; i <= lim; i++) {
14             if (!compuesto[i]) primos.push_back(i);
15             for (int p : primos) {
16                 if ((long long)i * p > lim) break;
17                 compuesto[i * p] = true;
18                 if (i % p == 0) break;
19             }
20         } // O(sqrt(R) log
21         log sqrt(R))
22
23         int sz = R - L + 1;
24         vector<long long> num(sz);
25         phi.assign(sz, 0);
26
27         for (int i = 0; i < sz; i++) {
28             num[i] = L + i;
29             phi[i] = L + i;
30
31             for (int p : primos) {
32                 long long start = ((L + p - 1) / p) * p;
33
34                 for (long long j = start; j <= R; j += p) {
35                     int idx = j - L;
36                     phi[idx] -= phi[idx] / p;
37
38                     while (num[idx] % p == 0)
39                         num[idx] /= p;
40                 }
41             }
42
43             // lo que quede > 1 es un primo grande (> sqrt(R)).
44             for (int i = 0; i < sz; i++)
45                 if (num[i] > 1)
46                     phi[i] -= phi[i] / num[i];
47
48             pref.assign(sz + 1, 0);
49             for (int i = 0; i < sz; i++)
50                 pref[i + 1] = pref[i] + phi[i]; // O((R-L+1) log
51             log R + sqrt(R))
52
53             // READ: suma de phi(i) para i en [l, r], con L <= l <= r <=
54             // R.
55             long long rango(long long l, long long r) {
56                 return pref[r - L + 1] - pref[l - L]; // O(1)
57             }
58
59             // READ: valor individual de phi(i), con L <= i <= R.
60             long long get(long long i) {
61                 return phi[i - L]; // O(1)
62             }
63 };

```

## Ejemplo de funcionamiento:

```

1 PhiSegmentedRangeSum ps(95, 105);
2 // calcula phi(95), phi(96), ..., phi(105)

```

```

3 // sin cribar desde 1
4
5 cout << ps.get(100);
6 // phi(100) = 40
7
8 cout << ps.rango(95, 105);
9 // suma de phi(95) a phi(105)
10
11 cout << ps.rango(100, 102);
12 // phi(100) + phi(101) + phi(102)

```

Cuál variante usar:

| Situación                      | Estructura           | Complejidad                         |
|--------------------------------|----------------------|-------------------------------------|
| $[1, N]$ , $N$ chico/mediano   | PhiRangeSum          | $O(N)$                              |
| $[L, R]$ grande, $R - L$ chico | PhiSegmentedRangeSum | $O((R - L) \log \log R + \sqrt{R})$ |
| $\phi$ cambia entre queries    | Fenwick sobre $\phi$ | $O(\log N)$                         |

Table 3: Variantes para la suma de  $\phi$ .

**Regla práctica:** el cálculo de  $\phi$  en sí (la criba, lineal o segmentada) es teoría de números y no cambia según la query. Una vez que tienes el arreglo de  $\phi$  listo, trátalo como *cualquier otro arreglo estático*: Prefix Sums resuelve el resto. No hace falta una estructura especial para  $\phi$  más allá de construir bien el arreglo inicial.

### 5.3 Árbol de Fenwick (BIT)

**Idea:** un arreglo normal da suma de prefijo en  $O(1)$  pero actualizar un elemento cuesta  $O(n)$  (o al revés si guardas prefijos). El Fenwick equilibra ambas a  $O(\log n)$  guardando *sumas parciales inteligentes*: la posición  $i$  almacena la suma de un bloque de tamaño  $i \& (-i)$  (su bit menos significativo) que termina en  $i$ . Para consultar un prefijo saltas hacia atrás quitando el bit bajo ( $i -= i \& (-i)$ ); para actualizar subes sumándolo ( $i += i \& (-i)$ ). Cada camino toca a lo más  $\log n$  posiciones (un bit por paso). Es 1-indexado.

**¿Por qué usarlo?** Es mucho más simple y liviano que un Segment Tree cuando solo necesitas sumas (o cualquier operación invertible) de prefijo/rango con actualizaciones puntuales. Menos memoria, menos código y una constante más pequeña.

**Casos de uso:**

- Suma de prefijo / suma de rango con updates frecuentes.
- Conteo de inversiones en un arreglo.
- Frecuencias acumuladas y búsqueda del  $k$ -ésimo elemento.
- Range update + range query cuando no hace falta un Segment Tree con Lazy Propagation.
- Consultas 2D: conteo/suma de puntos dentro de un rectángulo.

**Complejidad:** construir  $O(n)$ , update puntual  $O(\log n)$ , query de prefijo/rango  $O(\log n)$ , range update + range query  $O(\log n)$  y versión 2D  $O(\log n \cdot \log m)$ .

#### BIT clásico (Point Update, Range Query)

```

1 struct FenwickTree {
2     vector<long long> t;
3     int n;
4
5     // CREATE: reserva un arbol de tamaño n (1-indexado).

```

```

6     FenwickTree(int n) : t(n + 1, 0), n(n) {}
7
8     // CREATE: construye el arbol a partir de un arreglo a[0..n-1]
9     // propagando cada valor a su "padre" una sola vez, en vez de
10    // hacer n updates (que serian O(n log n)).
11    void build(vector<long long> &a) {
12        for (int i = 1; i <= n; i++) {
13            t[i] += a[i - 1];
14
15            int j = i + (i & (-i));
16
17            if (j <= n)
18                t[j] += t[i];
19        }
20    } // O(n)
21
22    // UPDATE: suma v al elemento en la posición i.
23    void update(int i, long long v) {
24        for (; i <= n; i += i & (-i))
25            t[i] += v;
26    } // O(log n)
27
28    // UPDATE: fija el valor absoluto de la posición i en x.
29    void set(int i, long long x) {
30        update(i, x - get(i));
31    } // O(log n)
32
33    // READ: suma de prefijo [1..i].
34    long long query(int i) {
35        long long s = 0;
36
37        for (; i > 0; i -= i & (-i))
38            s += t[i];
39
40        return s;
41    } // O(log n)
42
43    // READ: suma del rango [l, r].
44    long long rango(int l, int r) {
45        if (l > r)
46            return 0;
47
48        return query(r) - query(l - 1);
49    } // O(log n)
50
51    // READ: valor puntual en la posición i.
52    long long get(int i) {
53        return rango(i, i);
54    } // O(log n)
55
56    // SEARCH: menor índice i tal que query(i) >= objetivo.
57    // Usa binary lifting sobre el arbol.
58    // Requiere que todos los valores sean >= 0.
59    int lowerBound(long long objetivo) {
60        int pos = 0;
61        long long ac = 0;
62
63        int LOG = 0;
64
65        while ((1 << LOG) <= n)
66            LOG++;
67
68        for (int k = LOG - 1; k >= 0; k--) {
69            int nx = pos + (1 << k);
70
71            if (nx <= n && ac + t[nx] < objetivo) {
72                pos = nx;
73                ac += t[nx];
74            }
75        }
76
77        return pos + 1;
78    } // O(log n)
79
80    // DELETE: reinicia el arbol a ceros.
81    void clear() {
82        fill(t.begin(), t.end(), 0);
83    } // O(n)
84 };

```

#### Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 FenwickTree bit(5);
4 bit.build(a);
5
6 // arreglo logico:
7 // [5, 2, 9, 1, 7]
8 // posiciones 1..5
9
10 cout << bit.query(3);

```



```

11 // 5 + 2 + 9 = 16
12
13 cout << bit.rango(2, 4);
14 // 2 + 9 + 1 = 12
15
16 bit.update(2, 3);
17
18 // arreglo logico:
19 // [5, 5, 9, 1, 7]
20
21 cout << bit.get(2);
22 // 5
23
24 bit.set(4, 10);
25
26 // arreglo logico:
27 // [5, 5, 9, 10, 7]
28
29 cout << bit.lowerBound(15);
30 // menor indice cuyo prefijo alcanza 15
31 // -> indice 3 (5 + 5 + 9 = 19 >= 15)

```

## BIT con Range Update + Range Query (Doble BIT)

**Idea:** con *dos* Fenwick y el truco de diferencias se puede sumar  $v$  a todo un rango  $[l, r]$  en  $O(\log n)$  y también consultar la suma de cualquier rango en  $O(\log n)$ . Esto permite resolver simultáneamente actualizaciones y consultas de rango.

```

1 struct FenwickRangeUpdate {
2     FenwickTree b1, b2;
3     int n;
4
5     // CREATE: reserva dos arboles auxiliares de tamaño n.
6     FenwickRangeUpdate(int n) : b1(n), b2(n), n(n) {}
7
8     // UPDATE: suma v a todos los elementos del rango [l, r].
9     void rangoUpdate(int l, int r, long long v) {
10         b1.update(l, v);
11         b1.update(r + 1, -v);
12
13         b2.update(l, v * (l - 1));
14         b2.update(r + 1, -v * r);
15     }
16
17     // READ: suma de prefijo [1..i].
18     long long prefijo(int i) {
19         return b1.query(i) * i - b2.query(i);
20     }
21
22     // READ: suma del rango [l, r].
23     long long rango(int l, int r) {
24         if (l > r)
25             return 0;
26
27         return prefijo(r) - prefijo(l - 1);
28     }
29
30     // READ: valor puntual en la posición i.
31     long long get(int i) {
32         return rango(i, i);
33     }
34 };

```

### Ejemplo de funcionamiento:

```

1 FenwickRangeUpdate f(5);
2 // arreglo logico inicial: [0, 0, 0, 0, 0]
3
4 f.rangoUpdate(2, 4, 3);
5 // arreglo logico: [0, 3, 3, 3, 0]
6
7 cout << f.rango(1, 5);
8 // 0+3+3+3+0 = 9
9
10 f.rangoUpdate(1, 3, 2);
11 // arreglo logico: [2, 5, 5, 3, 0]
12
13 cout << f.get(2);
14 // 5

```

## BIT 2D (point update, sum de submatriz)

**Idea:** un BIT dentro de otro BIT. Cada update/query en  $x$  dispara un update/query completo en  $y$ , dando  $O(\log n \cdot \log m)$  en vez de  $O(nm)$  por consulta.

```

1 struct Fenwick2D {
2     vector<vector<long long>>> t;
3     int n, m;
4
5     // CREATE: reserva una matriz de arboles de tamaño n x m.
6     Fenwick2D(int n, int m) : t(n + 1, vector<long long>(m + 1,
7         0)),
8         n(n), m(m) {}
9
10    // UPDATE: suma v a la celda (x, y).
11    void update(int x, int y, long long v) {
12        for (int i = x; i <= n; i += i & (-i))
13            for (int j = y; j <= m; j += j & (-j))
14                t[i][j] += v;
15    }
16
17    // READ: suma de la submatriz [1..x] x [1..y].
18    long long query(int x, int y) {
19        long long s = 0;
20        for (int i = x; i > 0; i -= i & (-i))
21            for (int j = y; j > 0; j -= j & (-j))
22                s += t[i][j];
23        return s;
24    }
25
26    // READ: suma del rectangulo [x1..x2] x [y1..y2].
27    long long rango(int x1, int y1, int x2, int y2) {
28        return query(x2, y2) - query(x1 - 1, y2)
29            - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
30    }
31 };

```

### Ejemplo de funcionamiento:

```

1 Fenwick2D f(4, 4);
2
3 f.update(1, 1, 5);
4 f.update(2, 3, 7);
5 f.update(4, 4, 2);
6
7 cout << f.rango(1, 1, 2, 3);
8 // 5 + 7 = 12
9
10 cout << f.rango(3, 3, 4, 4);
11 // 2

```

### Diferencia con un arreglo de prefijos simple:

| Estructura         | Construir | Query              | Update             |
|--------------------|-----------|--------------------|--------------------|
| Prefijos simples   | $O(n)$    | $O(1)$             | $O(n)$             |
| FenwickTree        | $O(n)$    | $O(\log n)$        | $O(\log n)$        |
| FenwickRangeUpdate | $O(n)$    | $O(\log n)$        | $O(\log n)^*$      |
| Fenwick2D          | $O(nm)$   | $O(\log n \log m)$ | $O(\log n \log m)$ |

Table 4: Comparación de complejidades: Fenwick vs. prefijos simples.

\*Range update + range query, con la variante de doble BIT.

**Regla práctica:** si el arreglo cambia con frecuencia y solo necesitas sumas (o cualquier operación invertible) de prefijo o rango, el Fenwick da el mejor balance entre simplicidad de código y velocidad. Si necesitas min/max, operaciones no invertibles, o updates de rango con lazy propagation general, usa Segment Tree en su lugar.

## 5.4 Sqrt Decomposition

**Idea:** divide el arreglo en bloques de tamaño  $\approx \sqrt{n}$  y guarda un resumen (suma, mínimo, etc.) por bloque. Un update toca solo un bloque; una query de rango toca a lo más dos bloques *parciales* (recorridos elemento

por elemento) y varios bloques *completos* en el medio (usados de golpe con su resumen precalculado). Como hay  $\approx \sqrt{n}$  bloques y cada bloque tiene  $\approx \sqrt{n}$  elementos, todo cuesta  $O(\sqrt{n})$ : ni tan lento como un arreglo plano, ni tan rápido como Fenwick/Segment Tree, pero mucho más simple de escribir y de adaptar a operaciones raras.

**¿Por qué usarlo?** Es la estructura más flexible cuando no tienes tiempo de pensar un Segment Tree con lazy propagation correcto: soporta casi cualquier operación (suma, min, max, contar algo con una condición) con el mismo esqueleto de código, aunque sea  $O(\sqrt{n})$  en vez de  $O(\log n)$ . También es la base conceptual de Mo's Algorithm.

#### Casos de uso:

- Necesitas implementar algo rápido bajo presión, sin memorizar casos de Segment Tree con lazy propagation.
- Operaciones donde min/max cambian de forma que Fenwick no puede manejar (Fenwick solo sirve para operaciones invertibles como la suma).
- $n$  no es tan grande ( $\leq 10^5$ – $10^6$ ) y  $O(\sqrt{n})$  por operación es suficiente para el límite de tiempo.
- Base para Mo's Algorithm (ver esa sección).

**Complejidad:** construir  $O(n)$ , update puntual  $O(1)$  (suma) u  $O(\sqrt{n})$  (min/max), query de rango  $O(\sqrt{n})$ , range update + range query  $O(\sqrt{n})$  con lazy por bloque.

#### Sqrt Decomposition clásico (point update, range query — suma)

```

1 struct SqrtDecomposition {
2     vector<long long> a, bloque;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño -sqrt(n).
6
7     SqrtDecomposition(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloque.assign((n + tam - 1) / tam, 0);
13
14        for (int i = 0; i < n; i++)
15            bloque[i / tam] += a[i]; // O(n)
16    }
17
18    // UPDATE: suma v al elemento en la posición i.
19    void update(int i, long long v) {
20        a[i] += v;
21        bloque[i / tam] += v; // O(1)
22    }
23
24    // READ: suma del rango [l, r] (0-indexado, inclusivo).
25    long long rango(int l, int r) {
26        long long s = 0;
27        int bl = l / tam, br = r / tam;
28
29        if (bl == br) {
30            for (int i = l; i <= r; i++)
31                s += a[i];
32            return s;
33        }
34
35        for (int i = l; i < (bl + 1) * tam; i++)
36            s += a[i];
37
38        for (int b = bl + 1; b < br; b++)
39            s += bloque[b];
40
41        for (int i = br * tam; i <= r; i++)
42            s += a[i];
43    }
44 }

```

```

43         return s;
44     } // O(sqrt(n))
45 };

```

#### Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtDecomposition sd(a);
4 // arreglo lógico: [5, 2, 9, 1, 7, 3, 8, 4] (posiciones 0..7)
5 // tam = 2 -> bloques: [5+2, 9+1, 7+3, 8+4] = [7, 10, 10, 12]
6
7 cout << sd.rango(1, 5);
8 // 2 + 9 + 1 + 7 + 3 = 22
9
10 sd.update(3, 10);
11 // arreglo lógico: [5, 2, 9, 11, 7, 3, 8, 4]
12
13 cout << sd.rango(1, 5);
14 // 2 + 9 + 11 + 7 + 3 = 32

```

#### Sqrt Decomposition con range update + range query (lazy por bloque)

**Idea:** igual que arriba, pero cada bloque guarda un valor lazy pendiente que representa “sumar  $v$  a todo el bloque” sin tocar cada elemento individualmente. Solo se aplica a los elementos reales cuando un update o query **parcial** necesita tocar ese bloque específico.

```

1 struct SqrtRangeUpdate {
2     vector<long long> a, bloqueSum, lazy;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño -sqrt(n).
6
7     SqrtRangeUpdate(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        int nb = (n + tam - 1) / tam;
13        bloqueSum.assign(nb, 0);
14        lazy.assign(nb, 0);
15
16        for (int i = 0; i < n; i++)
17            bloqueSum[i / tam] += a[i]; // O(n)
18    }
19
20    // READ auxiliar: cantidad de elementos reales del bloque b.
21    int tamBloque(int b) {
22        return min(n, (b + 1) * tam) - b * tam; // O(1)
23    }
24
25    // UPDATE: suma v a todo el rango [l, r].
26    void rangoUpdate(int l, int r, long long v) {
27        int bl = l / tam, br = r / tam;
28
29        if (bl == br) {
30            for (int i = l; i <= r; i++)
31                a[i] += v;
32            bloqueSum[bl] += v * (r - l + 1);
33            return;
34        }
35
36        for (int i = l; i < (bl + 1) * tam; i++)
37            a[i] += v;
38        bloqueSum[bl] += v * ((bl + 1) * tam - l);
39
40        for (int b = bl + 1; b < br; b++)
41            lazy[b] += v;
42
43        for (int i = br * tam; i <= r; i++)
44            a[i] += v;
45        bloqueSum[br] += v * (r - br * tam + 1); // O(sqrt(n))
46    }
47
48    // READ: suma del rango [l, r].
49    long long rango(int l, int r) {
50        long long s = 0;
51        int bl = l / tam, br = r / tam;
52
53        if (bl == br) {
54            for (int i = l; i <= r; i++)
55                s += a[i] + lazy[bl];
56            return s;
57        }
58
59        for (int i = l; i < (bl + 1) * tam; i++)
60            s += a[i];
61
62        for (int b = bl + 1; b < br; b++)
63            s += bloqueSum[b] + lazy[b] * tam;
64
65        for (int i = br * tam; i <= r; i++)
66            s += a[i] + lazy[br] * (r - br * tam + 1);
67    }
68 }

```



```

58     for (int i = 1; i < (bl + 1) * tam; i++)
59         s += a[i] + lazy[bl];
60
61     for (int b = bl + 1; b < br; b++)
62         s += bloqueSum[b] + lazy[b] * tamBloque(b);
63
64     for (int i = br * tam; i <= r; i++)
65         s += a[i] + lazy[br];
66
67     return s;
68 }
69 // O(sqrt(n))

```

### Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SqrtRangeUpdate sr(a);
4 // tam = 2 -> bloques: [1+1, 1+1, 1+1] = [2, 2, 2]
5
6 sr.rangoUpdate(1, 4, 10);
7 // arreglo logico "efectivo": [1, 11, 11, 11, 11, 1]
8
9 cout << sr.rango(0, 5);
10 // 1+11+11+11+11+1 = 46
11
12 cout << sr.rango(2, 3);
13 // 11+11 = 22

```

### Variante para mínimo / máximo (point update, range query)

**Idea:** a diferencia de Fenwick (que solo sirve para operaciones *invertibles* como la suma), Sqrt Decomposition soporta min/max con el mismo esqueleto: la única diferencia es que un update ya no puede ajustar el resumen del bloque en  $O(1)$ , porque quitar el valor viejo no dice si sigue habiendo otro elemento igual de chico. Hay que **reconstruir el mínimo de todo el bloque**, lo cual sigue siendo  $O(\sqrt{n})$ .

```

1 struct SqrtMinQuery {
2     vector<long long> a, bloqueMin;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño -sqrt(n).
6
7     SqrtMinQuery(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloqueMin.assign((n + tam - 1) / tam, LLONG_MAX);
13
14        for (int i = 0; i < n; i++)
15            bloqueMin[i / tam] = min(bloqueMin[i / tam], a[i]);
16    }
17
18    // UPDATE: fija el valor absoluto de la posición i en x.
19    // Reconstruye el mínimo de todo el bloque, ya que a
20    // diferencia
21    // de la suma, el mínimo no se puede ajustar incrementalmente.
22
23    void set(int i, long long x) {
24        a[i] = x;
25
26        int b = i / tam;
27        long long mn = LLONG_MAX;
28
29        for (int j = b * tam; j < min(n, (b + 1) * tam); j++)
30            mn = min(mn, a[j]);
31
32        bloqueMin[b] = mn;
33    }
34    // O(sqrt(n))
35
36    // READ: mínimo del rango [l, r].
37    long long rango(int l, int r) {
38        long long mn = LLONG_MAX;
39        int bl = l / tam, br = r / tam;
40
41        if (bl == br) {
42            for (int i = l; i <= r; i++)
43                mn = min(mn, a[i]);
44            return mn;
45        }
46
47        for (int i = l; i < (bl + 1) * tam; i++)

```

```

44        mn = min(mn, a[i]);
45
46        for (int b = bl + 1; b < br; b++)
47            mn = min(mn, bloqueMin[b]);
48
49        for (int i = br * tam; i <= r; i++)
50            mn = min(mn, a[i]);
51
52        return mn;
53    }
54    // O(sqrt(n))

```

### Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtMinQuery sm(a);
4 // tam = 2 -> bloques min: [2, 1, 3, 4]
5
6 cout << sm.rango(0, 4);
7 // min(5,2,9,1,7) = 1
8
9 sm.set(3, 100);
10 // arreglo logico: [5, 2, 9, 100, 7, 3, 8, 4]
11
12 cout << sm.rango(0, 4);
13 // min(5,2,9,100,7) = 2

```

### Comparación con Fenwick y Segment Tree:

| Estructura         | Update               | Query         | Min/Max? |
|--------------------|----------------------|---------------|----------|
| Sqrt Decomposition | $O(1) - O(\sqrt{n})$ | $O(\sqrt{n})$ | Sí       |
| FenwickTree        | $O(\log n)$          | $O(\log n)$   | No       |
| Segment Tree       | $O(\log n)$          | $O(\log n)$   | Sí       |

Table 5: Sqrt Decomposition vs. Fenwick vs. Segment Tree.

**Regla práctica:** usa Sqrt Decomposition cuando quieras algo simple y flexible de escribir rápido, sobre todo si la operación no es una suma (Fenwick no sirve) y no quieres arriesgarte con un Segment Tree + Lazy Propagation bajo presión de tiempo. Si  $n$  es muy grande o el límite de tiempo es ajustado,  $O(\log n)$  de Segment Tree gana; si la operación es solo suma, Fenwick es más simple y más rápido.

## 5.5 Sparse Table (RMQ)

**Idea:** responde consultas de rango en  $O(1)$  sobre un arreglo que **no cambia**, precalculando  $sp[k][i]$  = el resultado de combinar el bloque que empieza en  $i$  y tiene largo  $2^k$ . La clave está en la consulta: para el rango  $[l, r]$  tomas  $k = \lfloor \log_2(r - l + 1) \rfloor$  y lo cubres con *dos* bloques de largo  $2^k$  —uno pegado a  $l$  y otro pegado a  $r$ — que **se solapan**. Si la operación es *idempotente* ( $f(x, x) = x$ , como min, max o gcd), contar el solape dos veces no cambia el resultado, así que la consulta son solo dos lecturas y una combinación. Para operaciones **no idempotentes** (como suma o xor) hace falta la variante *Disjoint Sparse Table*, que evita el solape por construcción.

**¿Por qué usarlo?** Es la estructura más rápida posible para consultas de rango ( $O(1)$  por query) cuando el arreglo es estático. A cambio de esa velocidad, no soporta actualizaciones: reconstruir cuesta  $O(n \log n)$ , así que si el arreglo cambia, conviene un Segment Tree.

### Casos de uso:

- RMQ clásico: mínimo o máximo de cualquier rango, arreglo estático.
- gcd o AND/OR de rango sobre un arreglo estático.

- LCA vía Euler Tour + RMQ  $\pm 1$ .
- Suma, xor o cualquier operación no idempotente de rango estático, en  $O(1)$ , usando Disjoint Sparse Table.
- Cualquier problema con muchísimas queries offline sobre datos que nunca se modifican.

**Complejidad:** construir  $O(n \log n)$ , consulta  $O(1)$ .  
(Si el arreglo *cambia*, usa Segment Tree:  $O(\log n)$  por operación.)

### Sparse Table clásico (operaciones idempotentes: min, max, gcd)

```
1 struct SparseTable {
2     vector<vector<long long>> sp;
3     vector<int> lg;
4     int n;
5
6     // CREATE: precalcula la tabla a partir de un arreglo a[0..n-1].
7     // sp[k][i] = mínimo del bloque [i, i + 2^k - 1].
8     SparseTable(vector<long long>& a) {
9         n = a.size();
10
11         lg.assign(n + 1, 0);
12         for (int i = 2; i <= n; i++)
13             lg[i] = lg[i / 2] + 1;
14
15         int K = lg[n] + 1;
16         sp.assign(K, vector<long long>(n));
17         sp[0] = a;
18
19         for (int k = 1; k < K; k++)
20             for (int i = 0; i + (1 << k) <= n; i++)
21                 sp[k][i] = min(sp[k - 1][i],
22                               sp[k - 1][i + (1 << (k - 1))]);
23     }
24
25     // READ: mínimo del rango [l, r] (0-indexado, inclusivo).
26     // Cubre [l, r] con dos bloques de largo 2^k que se solapan;
27     // como min es idempotente, el solape no afecta el resultado.
28     long long query(int l, int r) {
29         int k = lg[r - l + 1];
30         return min(sp[k][l], sp[k][r - (1 << k) + 1]);
31     }
32 };
```

#### Ejemplo de funcionamiento:

```
1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SparseTable st(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << st.query(0, 2);
7 // min(5, 2, 9) = 2
8
9 cout << st.query(2, 5);
10 // min(9, 1, 7, 3) = 1
11
12 cout << st.query(4, 4);
13 // min(7) = 7
```

**Nota:** para max o gcd solo cambia min(...) por max(...) o \_\_gcd(...) en el constructor y en query. Si necesitas saber *qué índice* alcanza el mínimo (no solo el valor), guarda índices en vez de valores y compara a[sp[k][i]] en lugar de sp[k][i].

### Disjoint Sparse Table (operaciones no idempotentes: suma, xor)

**Idea:** en vez de dos bloques que se solapan, cada nivel  $k$  particiona el arreglo en bloques de largo  $2^{k+1}$  y precalcula, para cada bloque, un prefijo hacia la izquierda desde su punto medio y un prefijo hacia la derecha desde

ese mismo punto medio. Toda consulta  $[l, r]$  cae exactamente en un nivel donde  $l$  y  $r$  quedan en mitades distintas del mismo bloque (el nivel lo da el bit más alto donde difieren  $l$  y  $r$ ), así que se combina sin solapar y sirve para operaciones **no idempotentes** como la suma.

```
1 struct DisjointSparseTable {
2     vector<vector<long long>> pre;
3     vector<long long> a;
4     int n, LOG;
5
6     // CREATE: precalcula prefijos izquierda/derecha por nivel,
7     // partiendo el arreglo en bloques de tamaño 2^(k+1).
8     DisjointSparseTable(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        LOG = 1;
12        while ((1 << LOG) < n) LOG++;
13        LOG++;
14
15        pre.assign(LOG, vector<long long>(n));
16
17        for (int k = 0; k < LOG; k++) {
18            int block = 1 << (k + 1);
19
20            for (int start = 0; start < n; start += block) {
21                int mid = min(start + block / 2, n);
22                int end = min(start + block, n);
23
24                if (mid > n) continue;
25
26                // prefijo hacia la izquierda desde mid-1
27                if (mid - 1 >= start) {
28                    pre[k][mid - 1] = a[mid - 1];
29                    for (int i = mid - 2; i >= start; i--)
30                        pre[k][i] = a[i] + pre[k][i + 1];
31                }
32
33                // prefijo hacia la derecha desde mid
34                if (mid < end) {
35                    pre[k][mid] = a[mid];
36                    for (int i = mid + 1; i < end; i++)
37                        pre[k][i] = pre[k][i - 1] + a[i];
38                }
39            }
40        }
41    }
42
43    // READ: suma del rango [l, r] (0-indexado, inclusivo).
44    // El nivel k lo da el bit mas alto donde l y r difieren:
45    // ahí es donde el bloque se parte en dos mitades disjuntas.
46    long long query(int l, int r) {
47        if (l == r) return a[l];
48
49        int k = 31 - __builtin_clz(1 ^ r);
50        return pre[k][l] + pre[k][r];
51    }
52 };
```

#### Ejemplo de funcionamiento:

```
1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 DisjointSparseTable dst(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << dst.query(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << dst.query(2, 5);
10 // 9 + 1 + 7 + 3 = 20
11
12 cout << dst.query(1, 1);
13 // 2
```

#### Diferencia entre las dos variantes:

| Estructura                       | Construir     | Query       | Updates         |
|----------------------------------|---------------|-------------|-----------------|
| Sparse Table (min/max/gcd)       | $O(n \log n)$ | $O(1)$      | No              |
| Disjoint Sparse Table (suma/xor) | $O(n \log n)$ | $O(1)$      | No              |
| Segment Tree                     | $O(n)$        | $O(\log n)$ | Sí, $O(\log n)$ |

Table 6: Sparse Table vs. Disjoint Sparse Table vs. Segment Tree.

**Regla práctica:** si el arreglo **no cambia**, usa Sparse Table para min/max/gcd, o Disjoint Sparse Table si necesitas suma, xor, o cualquier operación no idempotente.

En cuanto el arreglo **cambia** —aunque sea una sola actualización— reconstruir en  $O(n \log n)$  deja de valer la pena y conviene un Segment Tree.

## 5.6 Segment Tree

**Idea:** un árbol binario implícito (guardado en un arreglo) donde cada nodo representa el resultado combinado de un rango del arreglo original. La raíz cubre todo  $[0, n-1]$ ; cada nodo se parte en dos mitades hasta llegar a las hojas, que son elementos individuales. Un update baja por el árbol tocando  $O(\log n)$  nodos (uno por nivel); una query de rango se descompone en a lo más  $O(\log n)$  nodos que cubren exactamente el rango pedido, sin necesidad de bloques parciales como en Sqrt Decomposition. A diferencia de Fenwick, sirve para **cualquier** operación asociativa (min, max, gcd, and, or, no solo sumas), y con Lazy Propagation soporta actualizaciones de rango completo.

¿Por qué usarlo? Es la estructura más versátil de esta sección:  $O(\log n)$  tanto para update como para query, funciona con casi cualquier operación asociativa, y con lazy propagation resuelve range update + range query para operaciones que Fenwick no puede (min, max, asignar un valor a todo un rango, etc.).

### Casos de uso:

- Min/max/gcd de rango con actualizaciones (a diferencia de Sparse Table, que exige arreglo estático).
- Range update + range query con operaciones no invertibles: asignar un valor a todo un rango, min/-max con suma en rango, etc.
- Cualquier operación asociativa que no sea una suma simple (donde Fenwick ya alcanzaría y sería más simple).
- Base para variantes más avanzadas: Segment Tree 2D, Merge Sort Tree, Persistent Segment Tree, Segment Tree Beats.

**Complejidad:** construir  $O(n)$ , update puntual  $O(\log n)$ , query de rango  $O(\log n)$ , range update + range query con lazy propagation  $O(\log n)$ .

### Segment Tree clásico (point update, range query — suma)

```

1 struct SegmentTree {
2     vector<long long> tree, a;
3     int n;
4
5     // CREATE: reserva el arbol (tamano 4n es cota segura) y
6     // construye recursivamente a partir de a[0..n-1].
7     SegmentTree(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tree.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    } // O(n)
14
15    void build(int node, int l, int r) {
16        if (l == r) {
17            tree[node] = a[l];
18            return;
19        }
20
21        int mid = (l + r) / 2;
22        build(2 * node, l, mid);
23        build(2 * node + 1, mid + 1, r);

```

```

24        tree[node] = tree[2 * node] + tree[2 * node + 1];
25    }
26
27    void updateUtil(int node, int l, int r, int i, long long x) {
28        if (l == r) {
29            tree[node] = x;
30            return;
31        }
32
33        int mid = (l + r) / 2;
34        if (i <= mid)
35            updateUtil(2 * node, l, mid, i, x);
36        else
37            updateUtil(2 * node + 1, mid + 1, r, i, x);
38
39        tree[node] = tree[2 * node] + tree[2 * node + 1];
40    }
41
42    // UPDATE: fija el valor absoluto de la posicion i en x.
43    void update(int i, long long x) {
44        updateUtil(1, 0, n - 1, i, x);
45    } // O(log n)
46
47    long long queryUtil(int node, int l, int r, int ql, int qr) {
48        if (qr < l || r < ql)
49            return 0; // neutro para
50            suma
51
52        if (ql <= l && r <= qr)
53            return tree[node];
54
55        int mid = (l + r) / 2;
56        return queryUtil(2 * node, l, mid, ql, qr)
57            + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
58    }
59
60    // READ: suma del rango [l, r] (0-indexado, inclusivo).
61    long long rango(int l, int r) {
62        return queryUtil(1, 0, n - 1, l, r);
63    } // O(log n)
64 };

```

### Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 SegmentTree st(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << st.rango(1, 3);
7 // 2 + 9 + 1 = 12
8
9 st.update(2, 100);
10 // arreglo logico: [5, 2, 100, 1, 7]
11
12 cout << st.rango(1, 3);
13 // 2 + 100 + 1 = 103

```

**Nota:** para min/max/gcd solo cambia el neutro del caso base (LLONG\_MAX para min, 0 para gcd, etc.) y el operador de combinación (min(...), \_\_gcd(...)) en build, updateUtil y queryUtil.

### Segment Tree con Lazy Propagation (range update + range query — suma)

**Idea:** cada nodo guarda un valor lazy pendiente que representa “sumar  $v$  a todo este rango” sin aplicarlo todavía a los hijos. Se aplica (*se propaga*) solo cuando hace falta bajar más en el árbol para otra operación.

```

1 struct SegmentTreeLazyAdd {
2     vector<long long> tree, lazy, a;
3     int n;
4
5     // CREATE: reserva arbol + lazy, construye a partir de a[0..n-1].
6     SegmentTreeLazyAdd(vector<long long>& arr) {
7         a = arr;
8         n = a.size();
9         tree.assign(4 * n, 0);
10        lazy.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    } // O(n)
14
15    void build(int node, int l, int r) {
16        if (l == r) {

```

```

17     tree[node] = a[l];
18     return;
19 }
20
21 int mid = (l + r) / 2;
22 build(2 * node, l, mid);
23 build(2 * node + 1, mid + 1, r);
24 tree[node] = tree[2 * node] + tree[2 * node + 1];
25 }
26
27 // Aplica el lazy pendiente de este nodo y lo empuja a los
28 // hijos.
29 void propagar(int node, int l, int r) {
30     if (lazy[node] == 0)
31         return;
32
33     tree[node] += lazy[node] * (r - l + 1);
34
35     if (l != r) {
36         lazy[2 * node] += lazy[node];
37         lazy[2 * node + 1] += lazy[node];
38     }
39
40     lazy[node] = 0;
41 }
42
43 void rangoUpdateUtil(int node, int l, int r, int ql, int qr,
44                     long long v) {
45     propagar(node, l, r);
46
47     if (qr < l || r < ql)
48         return;
49
50     if (ql <= l && r <= qr) {
51         lazy[node] += v;
52         propagar(node, l, r);
53         return;
54     }
55
56     int mid = (l + r) / 2;
57     rangoUpdateUtil(2 * node, l, mid, ql, qr, v);
58     rangoUpdateUtil(2 * node + 1, mid + 1, r, ql, qr, v);
59     tree[node] = tree[2 * node] + tree[2 * node + 1];
60 }
61
62 // UPDATE: suma v a todo el rango [l, r].
63 void rangoUpdate(int l, int r, long long v) {
64     rangoUpdateUtil(1, 0, n - 1, l, r, v);
65 }
66
67 // O(log n)
68
69 long long queryUtil(int node, int l, int r, int ql, int qr) {
70     propagar(node, l, r);
71
72     if (qr < l || r < ql)
73         return 0;
74
75     if (ql <= l && r <= qr)
76         return tree[node];
77
78     int mid = (l + r) / 2;
79     return queryUtil(2 * node, l, mid, ql, qr)
80         + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
81 }
82
83 // READ: suma del rango [l, r].
84 long long rango(int l, int r) {
85     return queryUtil(1, 0, n - 1, l, r);
86 }
87
88 // O(log n)
89 }

```

### Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SegmentTreeLazyAdd st(a);
4
5 st.rangoUpdate(1, 4, 10);
6 // arreglo logico efectivo: [1, 11, 11, 11, 11, 1]
7
8 cout << st.rango(0, 5);
9 // 1+11+11+11+11+1 = 46
10
11 cout << st.rango(2, 3);
12 // 11+11 = 22

```

**Lazy Propagation con asignación (range assign + range query — mínimo)**

**Idea:** en vez de *sumar* un valor al rango, lo *reemplaza* por completo (asigna  $x$  a todo  $[l, r]$ ). El lazy aquí

no se acumula como en la suma: el último assign pisa a cualquier assign anterior pendiente, así que se necesita una bandera aparte para distinguir “no hay lazy pendiente” de “hay que asignar 0”.

```

1 struct SegmentTreeAssignMin {
2     vector<long long> tree, lazy, a;
3     vector<bool> tieneLazy;
4     int n;
5
6     // CREATE: reserva arbol + lazy + banderas, construye desde a.
7
8     SegmentTreeAssignMin(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        tree.assign(4 * n, LLONG_MAX);
12        lazy.assign(4 * n, 0);
13        tieneLazy.assign(4 * n, false);
14        if (n > 0)
15            build(1, 0, n - 1);
16    }
17
18    // O(n)
19
20    void build(int node, int l, int r) {
21        if (l == r) {
22            tree[node] = a[l];
23            return;
24        }
25
26        int mid = (l + r) / 2;
27        build(2 * node, l, mid);
28        build(2 * node + 1, mid + 1, r);
29        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
30    }
31
32    // Empuja el assign pendiente de este nodo hacia sus hijos.
33    void propagar(int node) {
34        if (!tieneLazy[node])
35            return;
36
37        for (int hijo : {2 * node, 2 * node + 1}) {
38            tree[hijo] = lazy[node];
39            lazy[hijo] = lazy[node];
40            tieneLazy[hijo] = true;
41        }
42
43        tieneLazy[node] = false;
44    }
45
46    void rangoAssignUtil(int node, int l, int r, int ql, int qr,
47                        long long x) {
48        if (qr < l || r < ql)
49            return;
50
51        if (ql <= l && r <= qr) {
52            tree[node] = x;
53            lazy[node] = x;
54            tieneLazy[node] = true;
55            return;
56        }
57
58        propagar(node);
59        int mid = (l + r) / 2;
60        rangoAssignUtil(2 * node, l, mid, ql, qr, x);
61        rangoAssignUtil(2 * node + 1, mid + 1, r, ql, qr, x);
62        tree[node] = min(tree[2 * node], tree[2 * node + 1]);
63    }
64
65    // UPDATE: asigna x a todo el rango [l, r].
66    void rangoAssign(int l, int r, long long x) {
67        rangoAssignUtil(1, 0, n - 1, l, r, x);
68    }
69
70    // O(log n)
71
72    long long queryUtil(int node, int l, int r, int ql, int qr) {
73        if (qr < l || r < ql)
74            return LLONG_MAX;
75
76        if (ql <= l && r <= qr)
77            return tree[node];
78
79        propagar(node);
80        int mid = (l + r) / 2;
81        return min(queryUtil(2 * node, l, mid, ql, qr),
82                  queryUtil(2 * node + 1, mid + 1, r, ql, qr));
83    }
84
85    // READ: mínimo del rango [l, r].
86    long long rango(int l, int r) {
87        return queryUtil(1, 0, n - 1, l, r);
88    }
89
90    // O(log n)
91 }

```

**Ejemplo de funcionamiento:**

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SegmentTreeAssignMin st(a);
4
5 cout << st.rango(0, 5);
6 // min(5,2,9,1,7,3) = 1
7
8 st.rangoAssign(0, 2, 100);
9 // arreglo logico efectivo: [100, 100, 100, 1, 7, 3]
10
11 cout << st.rango(0, 2);
12 // min(100,100,100) = 100
13
14 cout << st.rango(0, 5);
15 // min(100,100,100,1,7,3) = 1

```

**Comparación con las otras estructuras de rango:**

| Estructura          | Update             | Query             | Min/Max? |
|---------------------|--------------------|-------------------|----------|
| Segment Tree        | $O(\log n)$        | $O(\log n)$       | Sí       |
| Segment Tree + Lazy | $O(\log n)$ range  | $O(\log n)$ range | Sí       |
| FenwickTree         | $O(\log n)$ punto  | $O(\log n)$ rango | No       |
| Sqrt Decomposition  | $O(1)-O(\sqrt{n})$ | $O(\sqrt{n})$     | Sí       |

Table 7: Segment Tree vs. Fenwick vs. Sqrt Decomposition.

**Regla práctica:** usa Segment Tree cuando necesites min/max/gcd (operaciones no invertibles que Fenwick no soporta), o cuando necesites range update + range query con una operación que no sea suma simple. Si la operación es solo suma y no necesitas range update, Fenwick es más simple y más rápido en la práctica (menor constante). Si no confías en escribir el lazy propagation correcto bajo presión, Sqrt Decomposition es una alternativa más simple a costa de  $O(\sqrt{n})$  en vez de  $O(\log n)$ .

## 5.7 RMQ (Range Minimum Query)

**Idea:** RMQ es el problema general de "dado un arreglo, encuentra el mínimo en cualquier rango  $[l, r]$ ". Ya tienes las dos estructuras que lo resuelven: **Sparse Table** si el arreglo es estático ( $O(1)$  por query) y **Segment Tree** si cambia ( $O(\log n)$  por query). Esta sección no repite esas estructuras, sino que muestra las dos aplicaciones clásicas donde un problema que *no parece* de rango se puede **reformular** como RMQ sobre un arreglo auxiliar: LCA en árboles, y LCP entre sufijos arbitrarios de un string.

**¿Por qué usarlo?** Reconocer cuándo un problema es "RMQ disfrazado" es lo que realmente separa resolver estos problemas en  $O(1)-O(\log n)$  de resolverlos con fuerza bruta  $O(n)$  por consulta. El patrón se repite: construyes un arreglo auxiliar donde la respuesta a tu pregunta real equivale al mínimo de un rango, y le aplicas Sparse Table o Segment Tree normal.

**Casos de uso:**

- LCA (ancestro común más bajo) de dos nodos en un árbol, con muchas queries: Euler Tour + RMQ.
- LCP (prefijo común más largo) entre dos sufijos *cualquiera* de un string (no necesariamente consecutivos en el Suffix Array): Suffix Array + Kasai + RMQ sobre el LCP Array.
- Cualquier problema donde la respuesta se pueda expresar como "el mínimo de algo" sobre un rango de índices auxiliares.

**Complejidad:** en ambas aplicaciones, el precómputo (Euler Tour o Suffix Array + Kasai) es  $O(n)$  a  $O(n \log^2 n)$  según el método de Suffix Array usado; una vez armado el arreglo auxiliar, el RMQ es  $O(n \log n)$  de build y  $O(1)$  por query con Sparse Table (ambos casos son estáticos: el árbol y el string no cambian).

### LCA vía Euler Tour + RMQ (Sparse Table)

**Idea:** recorres el árbol con un Euler Tour, anotando el nodo visitado cada vez que **entras o vuelves** a él (arreglo de tamaño  $2n - 1$ ) junto con su profundidad. El LCA de  $u$  y  $v$  es el nodo de **menor profundidad** que aparece entre la primera aparición de  $u$  y la primera aparición de  $v$  en ese recorrido — exactamente un RMQ (por profundidad) sobre el arreglo del Euler Tour.

```

1 struct LCA {
2     vector<int> euler, primeraApar, profundidad;
3     vector<vector<int>> sp; // sp[k][i] = nodo con menor
4         profundidad
5     vector<int> lg;
6
7     // CREATE: construye el Euler Tour del arbol (arr. de tamaño
8     // 2n-1) y la Sparse Table sobre profundidad, para RMQ O(1).
9     LCA(vector<vector<int>>& adj, int raiz, int n) {
10         primeraApar.assign(n, -1);
11         profundidad.assign(n, 0);
12         euler.reserve(2 * n);
13
14         dfs(adj, raiz, -1, 0);
15         build(); // O(n log n)
16
17     void dfs(vector<vector<int>>& adj, int u, int p, int d) {
18         primeraApar[u] = euler.size();
19         profundidad[u] = d;
20         euler.push_back(u);
21
22         for (int v : adj[u]) {
23             if (v != p) {
24                 dfs(adj, v, u, d + 1);
25                 euler.push_back(u);
26             }
27         }
28     }
29
30     void build() {
31         int m = euler.size();
32         lg.assign(m + 1, 0);
33         for (int i = 2; i <= m; i++)
34             lg[i] = lg[i / 2] + 1;
35
36         int K = lg[m] + 1;
37         sp.assign(K, vector<int>(m));
38         sp[0] = euler;
39
40         for (int k = 1; k < K; k++)
41             for (int i = 0; i + (1 << k) <= m; i++) {
42                 int l = sp[k - 1][i];
43                 int r = sp[k - 1][i + (1 << (k - 1))];
44                 sp[k][i] = profundidad[l] < profundidad[r] ? l :
45                     r;
46             }
47
48         // READ: LCA de u y v.
49         int query(int u, int v) {
50             int l = primeraApar[u], r = primeraApar[v];
51             if (l > r) swap(l, r);
52
53             int k = lg[r - l + 1];
54             int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
55             return profundidad[a] < profundidad[b] ? a : b;
56         } // O(1)
57     };

```

**Ejemplo de funcionamiento:**

```

1 // arbol:
2 //      0
3 //     / \
4 //    1   2
5 //   / \
6 //  3   4
7 vector<vector<int>> adj(5);

```



```

8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 LCA lca(adj, 0, 5);
15
16 cout << lca.query(3, 4);
17 // 1 (padre comun de 3 y 4)
18
19 cout << lca.query(3, 2);
20 // 0 (raiz)
21
22 cout << lca.query(1, 4);
23 // 1 (uno es ancestro del otro)

```

## LCP entre dos sufijos arbitrarios (Suffix Array + Kasai + RMQ)

**Idea:** el LCP Array (via Kasai) solo te da el LCP entre sufijos **consecutivos** en el Suffix Array. Para el LCP entre dos sufijos **cualquiera**  $i, j$ : los ubicas por su rango en el SA ( $\text{rnk}[i]$ ,  $\text{rnk}[j]$ ) y el LCP entre ambos es el **mínimo** del LCP Array en el rango entre esas dos posiciones —otra vez RMQ, esta vez sobre el LCP Array.

```

1 struct SuffixLCPQuery {
2     vector<int> sa, rnk, lcp;
3     vector<vector<int>> sp;
4     vector<int> lg;
5     int n;
6
7     // CREATE: construye SA, LCP (Kasai) y Sparse Table sobre lcp,
8
9     // todo en un solo paso.
10    SuffixLCPQuery(const string& s) {
11        n = s.size();
12        sa = buildSuffixArray(s); // O(n log^2 n)
13        lcp = buildLCPArray(s, sa); // O(n), Kasai
14
15        rnk.assign(n, 0);
16        for (int i = 0; i < n; i++)
17            rnk[sa[i]] = i;
18
19        buildSparse(); // O(n log^2 n)
20    }
21
22    void buildSparse() {
23        lg.assign(n + 1, 0);
24        for (int i = 2; i <= n; i++)
25            lg[i] = lg[i / 2] + 1;
26
27        int K = lg[n] + 1;
28        sp.assign(K, vector<int>(n));
29        sp[0] = lcp;
30
31        for (int k = 1; k < K; k++)
32            for (int i = 0; i + (1 << k) <= n; i++)
33                sp[k][i] = min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
34    }
35
36    // READ: LCP entre el sufijo que empieza en i y el que
37    // empieza en j (0-indexado sobre el string original).
38    int lcpEntreSufijos(int i, int j) {
39        if (i == j) return n - i;
40
41        int a = rnk[i], b = rnk[j];
42        if (a > b) swap(a, b);
43
44        // el mínimo va de [a+1, b] en el LCP array
45        int k = lg[b - a];
46        return min(sp[k][a + 1], sp[k][b - (1 << k) + 1]); // O(1)
47    }
48 };

```

### Ejemplo de funcionamiento:

```

1 SuffixLCPQuery slq("banana");
2
3 cout << slq.lcpEntreSufijos(1, 3);
4 // sufijo desde 1: "anana"
5 // sufijo desde 3: "ana"
6 // LCP = "ana" -> 3
7
8 cout << slq.lcpEntreSufijos(0, 3);
9 // sufijo desde 0: "banana"

```

```

10 // sufijo desde 3: "ana"
11 // LCP = "" -> 0

```

**Nota:** `buildSuffixArray` y `buildLCPArray` son las funciones estándar de Suffix Array + Kasai (ver la sección de Algoritmos de Strings); aquí solo se reutilizan como precómputo antes de aplicar RMQ.

**Regla práctica:** cuando un problema pida "algo" entre dos elementos que no son adyacentes en su estructura original (dos nodos de un árbol, dos sufijos de un string, dos elementos de cualquier secuencia con una noción de "orden" y "profundidad/afinidad"), pregúntate si esa relación se puede expresar como el mínimo de un arreglo auxiliar construido con un recorrido lineal. Si sí, el problema se reduce a Sparse Table (estático) o Segment Tree (dinámico), ya cubiertos en las secciones anteriores.

## 5.8 Mo's Algorithm

**Idea:** responde  $q$  queries de rango **offline** (las conoces todas de antemano) en  $O((n + q)\sqrt{n})$  total, sin ninguna estructura de datos elaborada. La clave es **ordenar las queries** —no procesarlas en el orden dado— agrupándolas por bloque de tamaño  $\sqrt{n}$  según su extremo izquierdo  $l$ , y dentro de cada bloque por su extremo derecho  $r$  (alternando la dirección en bloques pares/impares para aprovechar mejor la localidad). Luego mantienes un puntero  $[curL, curR]$  que se **desliza** de una query a la siguiente con funciones `add/remove` en  $O(1)$  cada una. Como los punteros nunca retroceden más de lo necesario gracias al orden elegido, el total de movimientos amortizado es  $O((n + q)\sqrt{n})$  en vez de  $O(nq)$  con fuerza bruta.

**¿Por qué usarlo?** Sirve para respuestas que **no tienen una estructura de rango obvia** (por ejemplo: contar elementos distintos, sumar cuadrados de frecuencias, contar pares con cierta propiedad) donde escribir un Segment Tree específico sería complicado o imposible, pero sí es fácil mantener la respuesta incrementalmente al agregar/quitar un elemento a la vez.

### Casos de uso:

- Contar elementos distintos en un rango, con muchas queries.
- Sumas o conteos que dependen de *frecuencias* dentro del rango (no solo de valores individuales), donde `add/remove` son fáciles pero no hay forma clara de combinar dos sub-rangos con un Segment Tree.
- Todas las queries se conocen de antemano (offline). Si necesitas responder **online** (una query a la vez, sin ver las siguientes), Mo's no aplica: usa Segment Tree o Sqrt Decomposition.
- El arreglo tiene pocos updates puntuales intercalados con las queries: usa la variante *Mo's con updates* (3D Mo's).

**Complejidad:**  $O((n + q)\sqrt{n})$  total para  $q$  queries sin updates;  $O(n^{5/3})$  (aprox.) para la variante con updates, usando bloques de tamaño  $n^{2/3}$ .



## Mo's clásico (queries offline, sin updates)

**Ejemplo guía:** contar cuántos valores **distintos** hay en cada rango  $[l, r]$  preguntado.

```

1 struct MoAlgorithm {
2     int n, blockSize;
3     vector<int> a;
4     vector<long long> cnt;
5     int curL = 0, curR = -1;
6     long long curDistinct = 0;
7
8     // CREATE: reserva el arreglo de frecuencias segun el maximo
9     // valor posible, y fija el tamaño de bloque en  $\sqrt{n}$ .
10    MoAlgorithm(vector<int>& arr, int maxVal) {
11        a = arr;
12        n = a.size();
13        blockSize = max(1, (int)sqrt((double)n));
14        cnt.assign(maxVal, 0);
15    } // O(n)
16
17    // UPDATE: agrega el elemento en pos a la ventana actual.
18    void add(int pos) {
19        if (cnt[a[pos]]++ == 0)
20            curDistinct++;
21    } // O(1)
22
23    // UPDATE: quita el elemento en pos de la ventana actual.
24    void remove(int pos) {
25        if (--cnt[a[pos]] == 0)
26            curDistinct--;
27    } // O(1)
28
29    // READ: procesa todas las queries offline (l, r inclusive,
30    // 0-indexadas) y devuelve las respuestas en el orden
31    // original.
32    vector<long long> resolver(vector<array<int, 2>>& queries) {
33        int q = queries.size();
34        vector<int> idx(q);
35        iota(idx.begin(), idx.end(), 0);
36
37        sort(idx.begin(), idx.end(), [&](int i, int j) {
38            int bi = queries[i][0] / blockSize;
39            int bj = queries[j][0] / blockSize;
40            if (bi != bj) return bi < bj;
41            return (bi & 1) ? (queries[i][1] > queries[j][1])
42                : (queries[i][1] < queries[j][1]);
43        });
44
45        vector<long long> respuestas(q);
46        for (int id : idx) {
47            int l = queries[id][0], r = queries[id][1];
48
49            while (curR < r) add(++curR);
50            while (curL > l) add(--curL);
51            while (curR > r) remove(curR--);
52            while (curL < l) remove(curL++);
53
54            respuestas[id] = curDistinct;
55        }
56        return respuestas; // O((n+q) sqrt(n))
57    }

```

### Ejemplo de funcionamiento:

```

1 vector<int> a = {1, 2, 1, 3, 2, 4, 1};
2 // valores en [1, 4]
3
4 MoAlgorithm mo(a, 5);
5
6 vector<array<int, 2>> queries = {
7     {0, 2}, // [1, 2, 1] -> 2 distintos
8     {1, 4}, // [2, 1, 3, 2] -> 3 distintos
9     {0, 6} // arreglo completo -> 4 distintos
10 };
11
12 vector<long long> resp = mo.resolver(queries);
13 // resp = [2, 3, 4]

```

## Mo's con updates (3D Mo's)

**Idea:** si además de queries hay **updates** **puntuales** intercaladas, se agrega una tercera dimensión al ordenamiento: el *tiempo*  $t$  (cuántas updates ya ocurrieron antes de cada query). Se ordenan las queries por (bloque( $l$ ), bloque( $r$ ),  $t$ ), y además de deslizar

$[curL, curR]$ , se avanza o retrocede  $curT$  aplicando o revertiendo updates una por una. El bloque óptimo cambia a  $\approx n^{2/3}$  (en vez de  $\sqrt{n}$ ) porque ahora hay tres dimensiones que balancear.

```

1 struct MoUpdates {
2     int n, blockSize;
3     vector<int> a;
4     vector<long long> cnt;
5     vector<pair<int, int>> updates; // (pos, valor); ver nota
6     // abajo
7     int curL = 0, curR = -1, curT = 0;
8     long long curDistinct = 0;
9
10    // CREATE: reserva frecuencias y fija el bloque en  $n^{2/3}$ .
11    MoUpdates(vector<int>& arr, int maxVal) {
12        a = arr;
13        n = a.size();
14        blockSize = max(1, (int)cbrt((double)n) *
15            (int)cbrt((double)n));
16        cnt.assign(maxVal, 0);
17    } // O(n)
18
19    void add(int pos) {
20        if (cnt[a[pos]]++ == 0) curDistinct++;
21    } // O(1)
22
23    void remove(int pos) {
24        if (--cnt[a[pos]] == 0) curDistinct--;
25    } // O(1)
26
27    // CREATE: registra una update (aplicarla no es inmediato,
28    // solo queda en la lista para procesarse en su momento).
29    void agregarUpdate(int pos, int val) {
30        updates.push_back({pos, val});
31    } // O(1)
32
33    // UPDATE: aplica (o revierte, es la misma operacion) la
34    // update numero t. El truco: como updates[t].second guarda
35    // "el valor a poner", tras aplicarla queda guardado el valor
36    // viejo -> llamar de nuevo revierte exactamente al estado
37    // anterior.
38    void alternarUpdate(int t) {
39        int pos = updates[t].first;
40        int val = updates[t].second;
41
42        if (pos >= curL && pos <= curR)
43            remove(pos);
44
45        swap(a[pos], val);
46        updates[t].second = val;
47
48        if (pos >= curL && pos <= curR)
49            add(pos);
50    } // O(1)
51
52    // READ: procesa todas las queries offline (l, r, t) y
53    // devuelve las respuestas en el orden original.
54    vector<long long> resolver(vector<array<int, 3>>& queries) {
55        int q = queries.size();
56        vector<int> idx(q);
57        iota(idx.begin(), idx.end(), 0);
58
59        sort(idx.begin(), idx.end(), [&](int i, int j) {
60            int bi = queries[i][0] / blockSize;
61            int bj = queries[j][0] / blockSize;
62            if (bi != bj) return bi < bj;
63
64            int ri = queries[i][1] / blockSize;
65            int rj = queries[j][1] / blockSize;
66            if (ri != rj) return ri < rj;
67
68            return queries[i][2] < queries[j][2];
69        });
70
71        vector<long long> respuestas(q);
72        for (int id : idx) {
73            int l = queries[id][0], r = queries[id][1],
74                t = queries[id][2];
75
76            while (curR < r) add(++curR);
77            while (curL > l) add(--curL);
78            while (curR > r) remove(curR--);
79            while (curL < l) remove(curL++);
80
81            while (curT < t) alternarUpdate(curT++);
82            while (curT > t) alternarUpdate(--curT);
83
84            respuestas[id] = curDistinct;
85        }
86        return respuestas; // O(n^{5/3})

```

```

87 };
```

*aprox.*

**Ejemplo de funcionamiento:**

```

1 vector<int> a = {1, 2, 3};
2
3 MoUpdates mo(a, 5);
4 mo.agregarUpdate(0, 2); // update #0: a[0] = 2
5 mo.agregarUpdate(1, 1); // update #1: a[1] = 1
6
7 // queries: (l, r, cuantas updates ya se aplicaron)
8 vector<array<int, 3>> queries = {
9     {0, 2, 0}, // arreglo original [1,2,3] -> 3 distintos
10    {0, 2, 1}, // tras update #0: [2,2,3] -> 2 distintos
11    {0, 2, 2}  // tras update #1: [2,1,3] -> 3 distintos
12 };
13
14 vector<long long> resp = mo.resolver(queries);
15 // resp = [3, 2, 3]
```

**Nota:** para problemas sobre **árboles** (queries sobre un camino o subárbol) existe *Mo's on Trees*, que aplanan el árbol con un Euler Tour y aplica el mismo algoritmo sobre ese arreglo; el resto del razonamiento (ordenar queries, deslizar punteros) es idéntico.

#### Comparación con otras estructuras de rango:

| Situación                     | Estructura                  | Complejidad                 |
|-------------------------------|-----------------------------|-----------------------------|
| Queries offline, sin updates  | Mo's clásico                | $O((n+q)\sqrt{n})$          |
| Queries offline, con updates  | Mo's con updates            | $O(n^{5/3})$                |
| Queries online (una a la vez) | Segment Tree / Sqrt Decomp. | $O(\log n)$ / $O(\sqrt{n})$ |

Table 8: Cuándo usar Mo's Algorithm frente a estructuras online.

**Regla práctica:** usa Mo's solo cuando (1) todas las queries son offline, y (2) la respuesta es fácil de mantener incrementalmente con **add/remove**, pero difícil de combinar con un Segment Tree (min/max/suma si se combinan fácil, así que ahí Mo's no aporta nada). Si el problema admite Segment Tree o Fenwick directamente, esas opciones son más simples y no necesitan que las queries sean offline.

## 5.9 Wavelet Tree

**Idea:** es un árbol binario sobre el **rango de valores** del arreglo (no sobre las posiciones, como Segment Tree). La raíz cubre  $[lo, hi]$  de valores posibles; en cada nodo, los elementos con valor  $\leq$  mediana van al hijo izquierdo y el resto al derecho, **preservando su orden relativo** (partición estable). Cada nodo guarda un arreglo  $b[]$  con cuántos elementos, hasta cada posición, fueron hacia la izquierda. Con eso puedes, en cada nivel, saber exactamente qué sub-rango de posiciones le corresponde a cada hijo — y bajar por el árbol respondiendo preguntas sobre **valores dentro de un rango de posiciones** en  $O(\log C)$ , donde  $C$  es la cantidad de valores distintos.

**¿Por qué usarlo?** Es la única estructura de esta sección que responde eficientemente preguntas que combinan **posición y valor** a la vez: "el  $k$ -ésimo menor en  $[l, r]$ ", "cuántos elementos en  $[l, r]$  son  $\leq X$ ", "cuántas veces aparece exactamente  $X$  en  $[l, r]$ ". Ninguna de las estructuras anteriores (Fenwick, Segment Tree, Sparse Table) resuelve esto directamente sin re-plantear el problema.

#### Casos de uso:

- $k$ -ésimo elemento más chico (o más grande) dentro de un rango  $[l, r]$ .

- Conteo de elementos en  $[l, r]$  cuyo valor está en un rango  $[a, b]$ .
- Frecuencia de un valor específico dentro de un rango de posiciones (rank query).
- Arreglo **estático** — no soporta updates sin reconstruir (para eso existe la variante persistente, más avanzada).

**Complejidad:** construir  $O(n \log C)$ , cada query  $O(\log C)$ , donde  $C$  es la cantidad de valores distintos tras comprimir coordenadas.

#### Wavelet Tree: $k$ -ésimo, conteo $\leq X$ y frecuencia

```

1 struct WaveletTree {
2     int lo, hi;
3     WaveletTree *izq = nullptr, *der = nullptr;
4     vector<int> b; // b[i] = cuantos de a[0..i-1] fueron a la
5                     // izq.
6
7     // CREATE: construye recursivamente sobre el segmento [from,
8     // de un arreglo mutable, particionando por la mediana de
9     // valor
10    // [lo, hi]. Los valores deben venir ya comprimidos a [0, C
11    // -1].
12    WaveletTree(int* from, int* to, int lo, int hi) : lo(lo), hi(hi) {
13        if (lo == hi || from == to) return;
14
15        int mid = (lo + hi) / 2;
16        auto valZq = [mid](int x) { return x <= mid; };
17
18        b.reserve(to - from + 1);
19        b.push_back(0);
20        for (auto it = from; it != to; it++)
21            b.push_back(b.back() + valZq(*it));
22
23        auto pivote = stable_partition(from, to, valZq);
24        izq = new WaveletTree(from, pivote, lo, mid);
25        der = new WaveletTree(pivote, to, mid + 1, hi);
26    } // O(n log C)
27
28    // READ: k-esimo valor mas chico en [l, r] (1-indexado, k
29    // desde 1).
30    int kth(int l, int r, int k) {
31        if (l > r) return -1;
32        if (lo == hi) return lo;
33
34        int enIzq = b[r] - b[l - 1];
35        int lb = b[l - 1], rb = b[r];
36
37        if (k <= enIzq)
38            return izq->kth(lb + 1, rb, k);
39        return der->kth(l - lb, r - rb, k - enIzq);
40    } // O(log C)
41
42    // READ: cuantos elementos de [l, r] tienen valor <= val.
43    int contarMenorIgual(int l, int r, int val) {
44        if (l > r || val < lo) return 0;
45        if (hi <= val) return r - l + 1;
46
47        int lb = b[l - 1], rb = b[r];
48        return izq->contarMenorIgual(lb + 1, rb, val)
49            + der->contarMenorIgual(l - lb, r - rb, val);
50    } // O(log C)
51
52    // READ: cuantas veces aparece exactamente val en [l, r].
53    int contarIgual(int l, int r, int val) {
54        if (l > r || val < lo || val > hi) return 0;
55        if (lo == hi) return r - l + 1;
56
57        int mid = (lo + hi) / 2;
58        int lb = b[l - 1], rb = b[r];
59
60        if (val <= mid)
61            return izq->contarIgual(lb + 1, rb, val);
62        return der->contarIgual(l - lb, r - rb, val);
63    } // O(log C)
64
65    ~WaveletTree() {
66        delete izq;
67        delete der;
68    }
69 };
```

**Ejemplo de funcionamiento** (con compresión de coordenadas):

```

1 vector<int> original = {40, 10, 30, 10, 20, 40, 30};
2
3 // comprimir a [0, C-1] para que el wavelet tree trabaje con
4 // valores pequeños y contiguos.
5 vector<int> ordenado = original;
6 sort(ordenado.begin(), ordenado.end());
7 ordenado.erase(unique(ordenado.begin(), ordenado.end()), ordenado
8 .end());
9
10 vector<int> comprimido(ordenado.size());
11 for (int i = 0; i < (int)ordenado.size(); i++)
12     comprimido[i] = lower_bound(ordenado.begin(), ordenado.end(),
13     original[i]) - ordenado.begin();
14
15 // comprimido: [3, 0, 2, 0, 1, 3, 2] (valores 10,20,30,40 ->
16 // 0,1,2,3)
17
18 WaveletTree wt(comprimido.data(),
19     comprimido.data() + comprimido.size(),
20     0, (int)ordenado.size() - 1);
21
22 // 3er elemento mas chico en el rango [2, 6] (1-indexado)
23 int idx = wt.kth(2, 6, 3);
24 cout << ordenado[idx];
25 // posiciones 2..6 (1-index): 10,30,10,20,40,30 (valores 0-idx 2..6)
26
27 // -> valores reales: 30,10,20,40,30 ordenados: 10,20,30,30,40
28 // 3er mas chico = 30
29
30 cout << wt.contarMenorIgual(1, 7, 1);
31 // cuantos valores comprimidos <= 1 (o sea, <= 20) hay en todo el
32 // arreglo
33 // original: 40,10,30,10,20,40,30 -> <=20 son: 10,10,20 -> 3
34
35 cout << wt.contarIgual(1, 7, 0);
36 // cuantas veces aparece el valor comprimido 0 (o sea, 10) -> 2

```

**Comparación con las otras estructuras de rango:**

| Pregunta                                                                                  | Estructura                            |
|-------------------------------------------------------------------------------------------|---------------------------------------|
| Suma/min/max de rango                                                                     | Fenwick / Segment Tree / Sparse Table |
| $k$ -ésimo menor, o conteo por rango de <b>valores</b> , en un rango de <b>posiciones</b> | Wavelet Tree                          |
| Elementos distintos, moda, funciones no lineales de frecuencia                            | Mo's Algorithm                        |

Table 9: Cuándo el problema necesita Wavelet Tree en vez de las otras estructuras.

**Regla práctica:** si el enunciado combina una condición sobre **valores** (“menor que”, “entre  $a$  y  $b$ ”, “el  $k$ -ésimo más chico”) con un rango de **posiciones**, y el arreglo es estático, es casi siempre Wavelet Tree. Si solo te piden agregar (suma/min/max) sin condición sobre el valor, usa las estructuras más simples de las secciones anteriores.

## 5.10 Heavy-Light Decomposition

**Idea:** descompone un árbol en **cadenas** de forma que cualquier camino desde la raíz hasta cualquier nodo cruza a lo más  $O(\log n)$  cadenas distintas. En cada nodo, el hijo con el **subárbol más grande** (el hijo “pesado”) continúa la misma cadena que su padre; los demás hijos inician cadenas nuevas. Al numerar los nodos en el orden en que se visitan (DFS que siempre sigue primero al hijo pesado), cada cadena queda como un **rango contiguo** de posiciones — así que una query sobre un **camino** del árbol se convierte en  $O(\log n)$  queries de rango sobre un Segment Tree normal, una por cada cadena que el camino atraviesa.

**¿Por qué usarlo?** Es la forma estándar de llevar range queries de un arreglo a un **árbol**: sumas/mínimos/updates en el camino entre dos nodos, o en el subárbol de un nodo. Sin HLD, un camino arbitrario no tiene ninguna estructura contigua sobre la cual aplicar Fenwick o Segment Tree directamente.

**Casos de uso:**

- Suma/mínimo/máximo en el camino entre dos nodos  $u$  y  $v$ .
- Actualizar todos los nodos (o aristas) de un camino  $u-v$ .
- Consultas o updates sobre el **subárbol** completo de un nodo (más simple: el subárbol siempre es un rango contiguo, sin necesidad de descomponer en cadenas).
- Cualquier problema de árbol que “se sentiría” como Range Queries si el árbol fuera un arreglo lineal.

**Complejidad:** construir  $O(n)$  (dos DFS) más  $O(n)$  del Segment Tree subyacente; query o update de **camino**  $O(\log^2 n)$  ( $O(\log n)$  cadenas  $\times$   $O(\log n)$  por consulta al Segment Tree); query o update de **subárbol**  $O(\log n)$  (un solo rango contiguo).

**HLD: suma en camino, suma en subárbol**

**Idea de la implementación:** dos DFS iterativas (con pila explícita, para evitar stack overflow en árboles muy profundos o muy desbalanceados — un riesgo real con recursión cuando  $n \geq 10^5$  y el árbol es una cadena larga). El primero (`dfsTamano`) calcula el tamaño de cada subárbol y decide el hijo pesado de cada nodo. El segundo (`dfsDescomponer`) recorre siempre primero al hijo pesado (así la cadena sigue consecutiva en las posiciones) y solo después explora los hijos livianos, cada uno iniciando su propia cadena nueva. Las posiciones resultantes (`pos[]`) alimentan un Segment Tree normal —aquí se reutiliza `SegmentTreeLazyAdd` de la sección anterior— sobre el cual se hacen las queries reales.

```

1 struct HLD {
2     vector<vector<int>>> adj;
3     vector<int> padre, profundidad, tamano, pesado, cabeza, pos;
4     int n, timer = 0;
5     SegmentTreeLazyAdd* seg; // Segment Tree sobre el orden pos
6
7     // CREATE: dos DFS iterativas para calcular cadenas, luego
8     // arma
9     // el Segment Tree sobre el arreglo reordenado por pos[].
10    HLD(vector<vector<int>>& adjacency, vector<long long>&
11        valores,
12        int raiz) {
13        adj = adjacency;
14        n = adj.size();
15        padre.assign(n, -1);
16        profundidad.assign(n, 0);
17        tamano.assign(n, 1);
18        pesado.assign(n, -1);
19        cabeza.assign(n, raiz);
20        pos.assign(n, 0);
21
22        dfsTamano(raiz);
23        dfsDescomponer(raiz);
24
25        vector<long long> reordenado(n);
26        for (int v = 0; v < n; v++)
27            reordenado[pos[v]] = valores[v];
28
29        seg = new SegmentTreeLazyAdd(reordenado);
30    } // O(n)

```

```

29 // Fase 1: obtiene un orden donde cada padre aparece ANTES
30 // que
31 // sus hijos (equivalente a un preorden), usando una pila.
32 // Fase 2: recorre ese orden al revés (hijos antes que padres
33 // para acumular tamaño[] y decidir el hijo pesado, igual que
34 // haría la versión recursiva al "volver" de cada llamada.
35 void dfsTamano(int raiz) {
36     vector<int> orden;
37     stack<int> st;
38     st.push(raiz);
39     padre[raiz] = -1;
40     profundidad[raiz] = 0;
41
42     while (!st.empty()) {
43         int u = st.top();
44         st.pop();
45         orden.push_back(u);
46
47         for (int v : adj[u]) {
48             if (v != padre[u]) {
49                 padre[v] = u;
50                 profundidad[v] = profundidad[u] + 1;
51                 st.push(v);
52             }
53         }
54     }
55
56     for (int i = (int)orden.size() - 1; i >= 0; i--) {
57         int u = orden[i];
58         tamaño[u] = 1;
59         int maxSub = 0;
60
61         for (int v : adj[u]) {
62             if (v == padre[u]) continue;
63
64             tamaño[u] += tamaño[v];
65             if (tamaño[v] > maxSub) {
66                 maxSub = tamaño[v];
67                 pesado[u] = v;
68             }
69         }
70     }
71 } // O(n)
72
73 // Pila de pares (nodo, cabeza de su cadena). Para que el
74 // pesado quede en el tope de la pila (y por lo tanto se
75 // procese inmediatamente después, como en la versión
76 // recursiva), se empuja de ÚLTIMO: los hijos livianos
77 // (cadena nueva) se empujan primero.
78 void dfsDescomponer(int raiz) {
79     stack<pair<int, int>> st;
80     st.push({raiz, raiz});
81
82     while (!st.empty()) {
83         auto [u, h] = st.top();
84         st.pop();
85
86         cabeza[u] = h;
87         pos[u] = timer++;
88
89         for (int v : adj[u]) {
90             if (v == padre[u] || v == pesado[u]) continue;
91             st.push({v, v}); // cadena nueva
92         }
93
94         if (pesado[u] != -1)
95             st.push({pesado[u], h}); // misma cadena
96     }
97 } // O(n)
98
99 // READ: suma de todos los nodos en el camino u-v.
100 long long consultarCamino(int u, int v) {
101     long long resultado = 0;
102
103     while (cabeza[u] != cabeza[v]) {
104         if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
105             swap(u, v);
106
107         resultado += seg->rango(pos[cabeza[u]], pos[u]);
108         u = padre[cabeza[u]];
109     }
110
111     if (profundidad[u] > profundidad[v]) swap(u, v);
112     resultado += seg->rango(pos[u], pos[v]);
113     return resultado;
114 } // O(log^2 n)
115
116 // UPDATE: suma val a todos los nodos en el camino u-v.
117 void actualizarCamino(int u, int v, long long val) {
118     while (cabeza[u] != cabeza[v]) {

```

```

119         if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
120             swap(u, v);
121
122         seg->rangoUpdate(pos[cabeza[u]], pos[u], val);
123         u = padre[cabeza[u]];
124     }
125
126     if (profundidad[u] > profundidad[v]) swap(u, v);
127     seg->rangoUpdate(pos[u], pos[v], val);
128 } // O(log^2 n)
129
130 // READ: suma de todo el subarbol de u (rango contiguo por
131 // construcción, gracias al orden DFS de dfsDescomponer).
132 long long consultarSubarbol(int u) {
133     return seg->rango(pos[u], pos[u] + tamaño[u] - 1);
134 } // O(log n)
135
136 // UPDATE: suma val a todo el subarbol de u.
137 void actualizarSubarbol(int u, long long val) {
138     seg->rangoUpdate(pos[u], pos[u] + tamaño[u] - 1, val);
139 } // O(log n)
140 };

```

### Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //             /\
3 //            1 2 3
4 //             /\
5 //            4 5
6
7 vector<vector<int>> adj(6);
8 adj[0] = {1, 2, 3};
9 adj[1] = {0, 4, 5};
10 adj[2] = {0};
11 adj[3] = {0};
12 adj[4] = {1};
13 adj[5] = {1};
14
15 vector<long long> valores = {1, 2, 3, 4, 5, 6};
16 // nodo:      0 1 2 3 4 5
17
18 HLD hld(adj, valores, 0);
19
20 cout << hld.consultarCamino(4, 3);
21 // camino 4 -> 1 -> 0 -> 3 : valores 5+2+1+4 = 12
22
23 cout << hld.consultarSubarbol(1);
24 // subarbol de 1: nodos {1,4,5} : 2+5+6 = 13
25
26 hld.actualizarCamino(4, 5, 10);
27 // suma 10 a cada nodo del camino 4 -> 1 -> 5
28
29 cout << hld.consultarSubarbol(0);
30 // subarbol de 0 (todo el arbol) tras el update:
31 // 1+(2+10)+3+4+(5+10)+(6+10) = 51

```

**Nota:** al ser iterativas, `dfsTamano` y `dfsDescomponer` no tienen riesgo de stack overflow incluso en árboles con  $n \geq 10^5$  y forma de “cadena larga” (el peor caso para recursión). `SegmentTreeLazyAdd` es exactamente el struct de la sección de Segment Tree — cualquier otra variante (min, max, assign) funciona igual de bien como motor subyacente.

**Regla práctica:** usa HLD en cuanto el problema pida algo sobre un **camino** entre dos nodos de un árbol con muchas queries. Si solo pide algo sobre **subárboles** (sin caminos arbitrarios), no hace falta la descomposición en cadenas completa: basta un solo DFS para las posiciones (`pos[]` y `tamaño[]`) y un Segment Tree normal, ya que el subárbol siempre es un rango contiguo por sí solo.

## Parte III Grafos y Árboles

### 6 Grafos (Graphs)

Contenido en desarrollo.



## 7 Árboles (Trees)

Contenido en desarrollo.

## 8 Disjoint Set Union (DSU)

Contenido en desarrollo.

# Parte IV Ordenamiento y Búsqueda

## 9 Ordenamiento y Búsqueda (Sorting & Searching)

Varias búsquedas lineales y sub-lineales. (La búsqueda binaria tiene su propia sección.)

### 9.1 Número faltante (Missing Number)

**Idea:** los números  $0, 1, \dots, n$  deberían sumar  $\frac{n(n+1)}{2}$  (fórmula de Gauss). Si falta uno, la suma real queda corta *exactamente* en ese valor, así que el faltante es (suma esperada — suma real). Basta un recorrido para acumular la suma. Alternativa sin riesgo de overflow: hacer XOR de  $0..n$  y de todos los elementos; lo que quede es el faltante. **Complejidad:**  $O(n)$ .

```
1 long long esp = (long long)n*(n+1)/2; // suma 0..n
2 long long falta = esp - accumulate(a.begin(), a.end(), 0LL);
```

### 9.2 Máximo / Mínimo (Array Max/Min)

**Idea:** recorres el arreglo una sola vez llevando el menor y el mayor vistos hasta ahora; comparas cada elemento contra ambos y actualizas. No requiere ordenar (ordenar sería  $O(n \log n)$ , peor). Con un recorrido consigues los dos a la vez. **Complejidad:**  $O(n)$ .

```
1 int mn = *min_element(a.begin(), a.end());
2 int mx = *max_element(a.begin(), a.end());
```

### 9.3 Par de suma absoluta mínima

**Idea:** buscamos el par cuya suma esté *más cerca de cero*. Tras ordenar, ponemos un puntero en cada extremo. La suma de los extremos te dice hacia dónde moverte: si es negativa, para acercarla a 0 hay que subir el menor ( $lo++$ ); si es positiva, hay que bajar el mayor ( $hi--$ ). Cada movimiento descarta el candidato menos prometedor, y en el camino guardas el  $|suma|$  más pequeño visto. Los dos punteros recorren el arreglo una vez ( $O(n)$ ); domina el sort. **Complejidad:**  $O(n \log n)$ .

```
1 sort(a.begin(), a.end());
2 int lo=0, hi=a.size()-1; long long mej=LLONG_MAX;
3 while(lo<hi){ long long s=(long long)a[lo]+a[hi];
4   mej=min(mej, llabs(s));
5   if(s<0) lo++; else hi--; }
```

### 9.4 Par con diferencia dada (Difference Pair)

**Idea:** ¿existe un par cuya diferencia sea exactamente  $d$ ? Tras ordenar, dos punteros avanzan hacia adelante: si la diferencia actual  $a_j - a_i$  es menor que  $d$ , adelantas el puntero lejano ( $j$ ) para agrandarla; si es mayor, adelantas el cercano ( $i$ ) para achicarla; si es igual, encontraste el par. Como el arreglo está ordenado, la diferencia crece con  $j$  y decrece con  $i$ , así que el barrido nunca retrocede. **Complejidad:**  $O(n \log n)$  (domina el sort).

```
1 sort(a.begin(), a.end()); int i=0, j=1, n=a.size();
2 while(i<n && j<n){
3   if(i!=j && a[j]-a[i]==d) return true;
4   if(a[j]-a[i]<d) j++; else i++; }
```

### 9.5 Búsqueda ternaria (Ternary Search)

**Idea:** sirve para hallar el pico de una función **unimodal** (sube y luego baja, con un solo máximo — no para buscar un valor en un arreglo cualquiera). Partes el rango en tres con dos puntos  $m_1 < m_2$  y comparas: si  $f(m_1) < f(m_2)$ , el máximo no puede estar a la izquierda de  $m_1$ , así que descartas ese tercio; en caso contrario descartas el tercio derecho. Cada paso elimina  $\sim \frac{1}{3}$  del rango, por eso converge en logarítmico. (Para un mínimo, invierte la comparación.) **Complejidad:**  $O(\log n)$  — NO  $O(n \log n)$ .

```
1 while(hi-lo>2){ long long m1=lo+(hi-lo)/3, m2=hi-(hi-lo)/3;
2   if(f(m1)<f(m2)) lo=m1+1; else hi=m2-1; }
3 // revisa [lo,hi] y devuelve el x que maximiza f
```

### 9.6 Búsqueda de Fibonacci (Fibonacci Search)

**Idea:** misma estrategia que la binaria (descartar mitades de un arreglo ordenado), pero en vez de partir por el punto medio elige el punto de comparación usando números de Fibonacci consecutivos. Mantiene tres Fibonacci y en cada paso mira la posición  $off+f_2$ ; según el elemento sea menor o mayor que el objetivo, retrocede uno o dos pasos en la secuencia. Ventaja: solo usa *sumas y restas*, nunca división, lo que la hace útil en hardware sin división rápida o para saltar por posiciones amigables con la caché. **Complejidad:**  $O(\log n)$ .

```
1 int f2=0, f1=1, fb=f1+f2;
2 while(fb<n){ f2=f1; f1=f2; fb=f1+f2; }
3 int off=-1;
4 while(fb>1){ int i=min(off+f2, n-1);
5   if(a[i]<x){ fb=f1; f1=f2; f2=fb-f1; off=i; }
6   else if(a[i]>x){ fb=f2; f1=f1-f2; f2=fb-f1; }
7   else return i; }
8 if(f1==1 && off+1<n && a[off+1]==x) return off+1;
```

(En Java es idéntico al C++, cambiando `a.length`.)

### 9.7 Búsqueda exponencial (Exponential Search)

**Idea:** dos fases sobre un arreglo **ordenado**. Primero *acotas* el objetivo doblando el índice ( $1, 2, 4, 8, \dots$ ) hasta que  $a[i]$  lo supera; en ese momento sabes que el objetivo está entre  $i/2$  e  $i$ . Segundo, haces una binaria normal dentro de ese bloque. Como el bloque encontrado tiene tamaño proporcional a la posición del objetivo, es ideal cuando el arreglo es enorme o de tamaño desconocido y el objetivo está cerca del inicio. **Complejidad:**  $O(\log n)$ .

```
1 if(a[0]==x) return 0;
2 int i=1; while(i<n && a[i]<=x) i*=2; // dobla el rango
3 // luego binaria en [i/2, min(i, n-1)]
```



## 9.8 Búsqueda por saltos (Jump Search)

**Idea:** en vez de revisar elemento por elemento, avanzas en bloques de tamaño  $\sqrt{n}$  mirando solo el *último* de cada bloque; cuando ese último ya supera al objetivo, sabes que (si existe) está en el bloque anterior, así que lo barres linealmente. El balance  $\sqrt{n}$  saltos +  $\sqrt{n}$  del barrido minimiza el total. Punto medio entre la lineal ( $O(n)$ ) y la binaria ( $O(\log n)$ ), útil cuando saltar hacia atrás es costoso. **Complejidad:**  $O(\sqrt{n})$ .

```
1 int paso=max(1,(int)sqrt((double)n)), prev=0;
2 while(prev<n && a[min(prev+paso,n)-1]<x) prev+=paso;
3 for(int i=prev; i<min(prev+paso,n); i++)
4   if(a[i]==x) return i;
```

## 9.9 Heap Sort

**Idea:** un *heap* binario se representa en el mismo arreglo: el hijo izquierdo de  $i$  es  $2i+1$  y el derecho  $2i+2$ ; en un *max-heap* todo padre es  $\geq$  sus hijos, así que el mayor está en la raíz. Heap Sort tiene dos fases. (1) **Construir el heap:** recorres de abajo hacia arriba aplicando *bajar* (sift-down) a cada padre. Aunque parezca  $O(n \log n)$ , es  $O(n)$ : casi todos los nodos están cerca de las hojas y se hunden poco (la suma  $\sum n/2^h \cdot h$  converge a  $2n$ ). (2) **Ordenar:** repetidamente intercambias la raíz (el mayor) con el último elemento activo —quedando ya ordenado al final— y “hundes” la nueva raíz para restaurar el heap; cada una de las  $n$  extracciones cuesta  $O(\log n)$ . In-place, sin memoria extra, pero **no estable**. **Complejidad:**  $O(n \log n)$  en todos los casos.

```
1 void bajar(vector<int>&a,int n,int i){ // hunde: O(log n)
2   while(true){ int m=i,l=2*i+1,r=2*i+2;
3     if(l<n&&a[l]>a[m]) m=l;
4     if(r<n&&a[r]>a[m]) m=r;
5     if(m==i) break; swap(a[i],a[m]); i=m; } }
6 void heapSort(vector<int>&a){ int n=a.size();
7   for(int i=n/2-1;i>=0;i--) bajar(a,n,i); // heap: O(n)
8   for(int i=n-1;i>0;i--){ swap(a[0],a[i]); // max al final
9     bajar(a,i,0); } }
```

(En Java es idéntico al C++ con `a.length()`.)

## 9.10 QuickSelect

**Idea:** encuentra el  $k$ -ésimo menor *sin ordenar todo*. Como quicksort, particiona alrededor de un pivote dejando a la izquierda lo menor y a la derecha lo mayor; pero como sabes en qué lado cae la posición  $k$ , recorres (o iteras) sobre *un solo* lado y descartas el otro. Por eso el promedio baja de  $O(n \log n)$  a  $O(n)$ : cada fase procesa *la mitad* del anterior, y  $n + \frac{n}{2} + \frac{n}{4} + \dots = 2n$ . El peor caso  $O(n^2)$  aparece con pivotes pésimos (arreglo casi ordenado + pivote extremo); **elegir el pivote al azar** lo vuelve prácticamente imposible. La versión de abajo es iterativa (sin pila de recursión). Si necesitas garantía  $O(n)$  en el peor caso, existe *mediana de medianas* para elegir un buen pivote. **Complejidad:** promedio  $O(n)$ , peor  $O(n^2)$ .

```
1 int quickSelect(vector<int> a, int k){ // k in [0,n-1]
2   int lo=0, hi=a.size()-1;
3   while(lo<hi){
4     int p=a[lo+rand()%(hi-lo+1)], i=lo, j=hi; // pivote aleatorio
5     while(i<=j){ while(a[i]<p) i++; while(a[j]>p) j--;
6       if(i<=j){ swap(a[i],a[j]); i++; j--; } }
7     if(k<=j) hi=j; // k a la izq
8     else if(k>=i) lo=i; // k a la der
9     else return a[k];
10  }
```

```
11 return a[lo];
12 }
13 // C++ trae nth_element(a.begin(), a.begin()+k, a.end()); // O(n)
```

(En Java igual, con `Random`; código completo en el repo.)

## 10 Búsqueda Binaria (Binary Search)

**Idea central:** mantienes un rango  $[lo, hi]$  donde *sabes* que está la respuesta y en cada paso lo partes por la mitad, descartando la mitad que no puede contenerla. Como reduces a la mitad cada vez, llegas en  $\log_2 n$  pasos. Funciona si los datos están **ordenados** o, más general, si existe un **predicado monótono** (una condición que pasa de falsa a verdadera una sola vez): entonces buscas la frontera del cambio. **Complejidad:**  $O(\log n)$  en discreto,  $O(\text{iter})$  en reales,  $O(L \log n)$  con cadenas. Tip: usa  $lo+(hi-lo)/2$  para el `mid` y evitar overflow.

### 10.1 En arreglo ordenado

**Cómo:** comparas  $x$  con el elemento del medio; si es menor, descartas la mitad derecha, si es mayor la izquierda. `lower_bound` te da el primer índice con valor  $\geq x$  y `upper_bound` el primer  $> x$ ; con eso sabes si  $x$  existe y cuántas copias hay.

```
1 int i = lower_bound(a.begin(),a.end(),x)-a.begin(); // >= x
2 int j = upper_bound(a.begin(),a.end(),x)-a.begin(); // > x
3 bool hay = binary_search(a.begin(),a.end(),x);
```

### 10.2 Menor $x$ que cumple

**Cómo:** aquí no buscas en un arreglo sino en el *rango de respuestas posibles*. El predicado es monótono F..F T..T (una vez que algo “sirve”, todo lo mayor también). En cada paso pruebas el `mid`: si cumple, lo guardas como candidato y sigues buscando a la izquierda (`hi=mid-1`) por si hay uno menor que también sirva; si no cumple, subes (`lo=mid+1`). Al final `res` es la frontera izquierda: el *menor* que cumple. Ejemplo: “mínima capacidad para repartir en  $\leq k$  días”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; hi=mid-1;} // sirve -> izq
4   else lo=mid+1; }
```

### 10.3 Mayor $x$ que cumple

**Cómo:** el caso espejo. Ahora el predicado es T..T F..F (sirve hasta cierto punto y luego deja de servir). Cuando el `mid` cumple, lo guardas y buscas a la *derecha* (`lo=mid+1`) por si hay uno mayor que también sirva; si no cumple, bajas (`hi=mid-1`). Respecto al anterior solo cambian esas dos líneas. Ejemplo: “máxima longitud de corte tal que salgan  $\geq k$  piezas”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; lo=mid+1;} // sirve -> der
4   else hi=mid-1; }
```

## 10.4 Sobre reales

**Cómo:** cuando la respuesta es un número *real* (raíces, puntos de equilibrio) no hay “mitad entera” ni condición  $lo \leq hi$  para parar. En su lugar iteras un número fijo de veces (unas 100 basta: el rango se divide por  $2^{100}$ , precisión más que suficiente), moviendo  $lo$  o  $hi$  al  $mid$  según el predicado. Al terminar,  $lo \approx hi$  es la frontera buscada.

```
1 for(int it=0; it<100; it++){ double mid=(lo+hi)/2;
2   if(cumple(mid)) hi=mid; else lo=mid; }
3 // lo ~ hi = frontera
```

## 10.5 Con strings

**Cómo:** exactamente la misma binaria, pero comparando cadenas en orden *lexicográfico* (diccionario) en lugar de números. La única diferencia de costo es que comparar dos cadenas cuesta  $O(L)$  (longitud), no  $O(1)$ , así que el total sube a  $O(L \log n)$ . Ojo: el arreglo debe estar ordenado con el mismo criterio (ASCII: mayúsculas antes que minúsculas).

```
1 // vector<string> s ordenado
2 int i = lower_bound(s.begin(), s.end(), string("caro"))
3   - s.begin();
4 bool hay = binary_search(s.begin(), s.end(), string("caro"));
```

# Parte V Matemáticas

## 11 Matemáticas (Math)

*Contenido en desarrollo.*

## 12 Teoría de Números (Number Theory)

*Contenido en desarrollo.*

## 13 Combinatoria (Combinatorics)

*Contenido en desarrollo.*

## 14 Máscaras de Bits (Bitmasking)

*Contenido en desarrollo.*

## 15 Geometría (Geometry)

*Contenido en desarrollo.*

# Parte VI Técnicas Algorítmicas

## 16 Programación Dinámica (Dynamic Programming)

*Contenido en desarrollo.*

## 17 Algoritmos Voraces (Greedy)

*Contenido en desarrollo.*

## 18 Ad-hoc y Simulación

*Contenido en desarrollo.*

# Parte VII Cadenas

## 19 Cadenas (Strings)

*Contenido en desarrollo.*